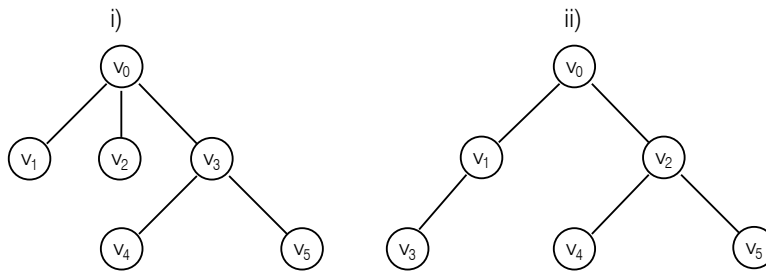


Es muy usual dibujar los árboles dirigidos con raíz v_0 o simplemente árboles con raíz v_0 cambiando flechas por aristas pero conservando el sentido de las flechas en forma implícita. Es decir, aún cuando no estén dibujados los arcos, los árboles siguen siendo árboles con raíz. Así podemos redibujar la Figura 6.9 como sigue:

Figura 6.10



Sin embargo, también es posible estudiar árboles no dirigidos (o simplemente árboles).

Definición 6.40

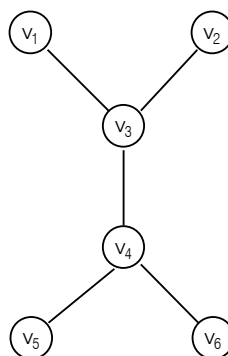
Para el caso *no dirigido* definimos el *árbol* como un *grafo conexo y sin ciclos*.

Observa la figura 6.10 ii) y cuenta el número de vértices y el número de aristas. ¿Qué relación existe entre ellos?

Ejemplo 22

La Figura 6.11 es un árbol, donde $V = \{v_1, v_2, v_3, v_4, v_5, v_6\}$ es el conjunto de nodos. Observándola notamos que un árbol no tiene lazos; entre dos vértices existe una única cadena que los conecta y que el número de aristas es igual al número de vértices menos uno.

Figura 6.11



Si quisiéramos pasar desde un *árbol no dirigido* hacia otro *árbol dirigido* notaremos que un árbol no dirigido corresponderá, en general, a varios árboles dirigidos, porque se pierde el concepto de raíz única. Al intentar buscar un vértice como raíz, vemos que cualquier vértice puede asumir dicho rol.

Ejemplo 23

A partir del siguiente árbol no dirigido (como grafo conexo y sin ciclos), podemos obtener distintos árboles con raíz, hecho que depende de la raíz escogida. Para el árbol de la Figura 6.12, representamos un árbol con raíz " v_2 " (Figura 6.13 i)) y otro árbol donde la raíz es " v_7 " (Figura 6.13 ii)).

Figura 6.12

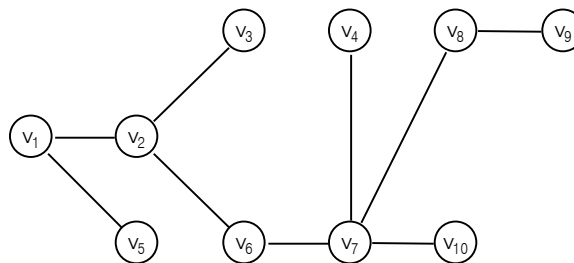
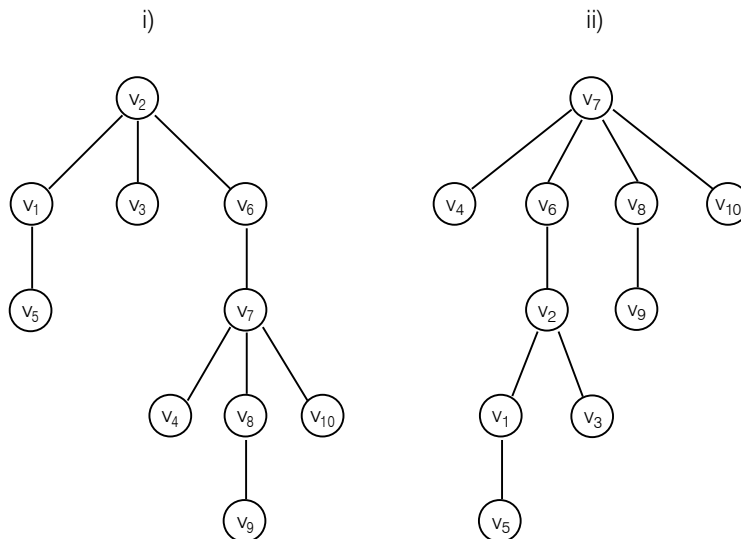


Figura 6.13



A continuación enunciaremos dos propiedades referidas a árboles no dirigidos.

Propiedad 6.7

Sea $T = \{V, A, \varphi\}$ un grafo y $o(V) = n$. Son equivalentes:

- i) T es un árbol.
- ii) T no tiene ciclos y $o(A) = n - 1$.
- iii) T es conexo y $o(A) = n - 1$.
- iv) T es acíclico y agregando una arista entre vértices no adyacentes se crea un ciclo y sólo uno.
- v) T es conexo y suprimiendo una arista cualquiera deja de serlo.

Propiedad 6.8

Todo árbol con $o(V) > 1$ tiene al menos dos vértices pendientes.

Demostración:

Se escoge un vértice v cualquiera y se hace un *recorrido* del árbol. Esto significa que se comienza con un vértice, continuando con una arista, luego otro vértice y se sigue con una arista no utilizada. Este camino no puede volver a pasar por un vértice ya utilizado porque sino tendría un ciclo, por lo que terminará en un vértice pendiente.

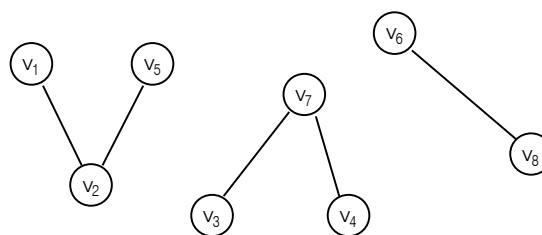
Seleccionando este vértice pendiente se comienza con un nuevo recorrido del árbol; como el recorrido no repite aristas y no se puede volver a pasar por un vértice ya utilizado el proceso debe terminar en un vértice final que es uno pendiente.

Definición 6.41

Si un grafo es no conexo y cada componente conexa es un árbol, el grafo se denomina *bosque*.

Ejemplo 24

Si $V = \{v_1, v_2, v_3, v_4, v_5, v_6, v_7, v_8\}$, el siguiente grafo es un ejemplo de bosque:



Los árboles con raíz, importantes en ciencias de la computación para organizar, buscar y ordenar datos, fueron ya definidos en el comienzo de la sección.

A continuación vamos a identificar mediante nombres a sus vértices:

Definición 6.42

Sea $T = (V, A, \varphi)$ un árbol con raíz. Una *hoja* (o *vástago*) es cualquier vértice que posea grado positivo igual a cero.

Definición 6.43

Sea $T = (V, A, \varphi)$ un árbol con raíz. Un *vértice interno* (o *vértice de ramificación*) es cualquier vértice que no sea hoja.

Ejemplo 25

En el árbol con raíz de la Figura 6.9 i) los vértices v_1, v_2, v_4 y v_5 son hojas y los vértices v_0 y v_3 son internos.

Actividad 6

Problema N°1: Dibuja, si es posible, el árbol dirigido T , que corresponde a cada uno de los enunciados dados o explica por qué no existe tal árbol:

- 1.1) T tiene cuatro vértices de grado 3, dos de grado 5, dos de grado 4, otro de grado dos y dieciséis de grado 1.
- 1.2) T tiene seis arcos y cinco vértices.
- 1.3) T tiene todos los vértices con grado negativo igual a 1.
- 1.4) T posee seis vértices de grados 1, 1, 1, 1, 2, 3.
- 1.5) T tiene todos los vértices de grado positivo igual a tres.
- 1.6) T posee seis vértices de grados: 1, 1, 2, 2, 1, 1.
- 1.7) T tiene seis vértices de grados: 1, 1, 1, 1, 3, 3.

Problema N°2: Sea $T = (V, A, \varphi)$ un árbol con raíz donde todos los vértices internos tienen grado positivo m , $o(V) = n$, con h hojas y son i los vértices internos.

Demuestra o justifica las siguientes igualdades:

- 2.1) $n = m i + 1$
- 2.2) $h = (m - 1) i + 1$
- 2.3) $i = (h - 1) / (m - 1) = (n - 1) / m$.
- 2.4) $n = (m h - 1) / (m - 1)$
- 2.5) $h = [(m - 1) n + 1] / m$

Problema N°3:

3.1) Determina el número de vértices y aristas para el siguiente árbol dirigido T , donde: T tiene $2n$ vértices de grado 1, $3n$ vértices de grado 2 y n vértices de grado 3.

3.2) Determina el número de vértices, hojas y aristas para los siguientes árboles dirigidos:

- a) T es un árbol donde los vértices que no son hojas tienen grados 1, 2, 2, 3, 4, 4, 5.
- b) T tiene 15 vértices internos de grado positivo igual a 4.

Problema N°4: Las siguientes matrices corresponden a matrices de adyacencia de árboles dirigidos.

$$M_1 = \begin{bmatrix} 0 & 1 & 1 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix} \quad M_2 = \begin{bmatrix} 0 & 1 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 \end{bmatrix}$$

- 4.1) ¿Qué significa que una columna tenga todos sus elementos nulos?
- 4.2) ¿Puede tener dos columnas nulas?
- 4.3) ¿Qué significa que una fila tenga todos sus elementos nulos?
- 4.4) ¿Puede tener dos filas nulas?
- 4.5) Esquematiza cada uno de ellos teniendo en cuenta todos los datos aportados por estas matrices.

Problema N°5: Demuestra que la suma de los grados de los n vértices de un árbol es $2n - 2$.

Problema N°6: Un árbol tiene dos vértices de grado 2, un vértice de grado 3 y tres vértices de grado 4. ¿Cuántos vértices de grado 1 tendrá el árbol?

Problema N°7: Un *árbol de expansión* de un grafo conexo $G = (V, A, \varphi)$ es un grafo que es conexo y acíclico, que tiene a todo el conjunto V como conjunto de nodos y a un subconjunto de A como conjunto de aristas. Es decir, un árbol de expansión es un grafo parcial del dado, que es conexo y acíclico.

Dado el grafo de la Figura 6.14 siguiente, hemos generado distintos árboles de expansión correspondientes al mismo grafo (Figuras 6.15 i) y ii)).

Figura 6.14

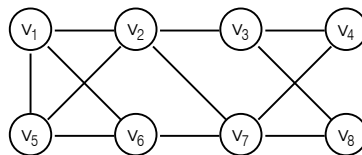
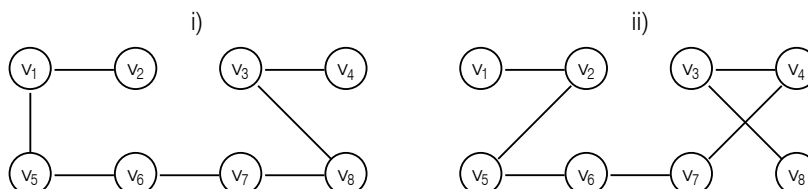
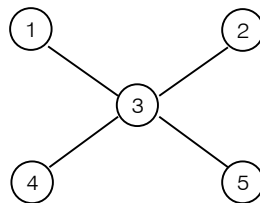


Figura 6.15



Dibuja otro árbol de expansión distinto a los dados para la Figura 6.14.

Problema N°8: Dado el siguiente árbol no dirigido, muestra todos los árboles con raíz que puedas generar. ¿Cuántos existen?



6.9 Árboles con raíz y definiciones recursivas

En las ciencias de la computación a menudo se trabaja sobre definiciones recursivas para las estructuras de árboles. Muchas operaciones sobre éstos pueden simplificarse considerando la recursividad de los mismos. Para abordar esta nueva propuesta, previamente introduciremos una terminología propia, teniendo en cuenta que en este enfoque es probable encontrar definiciones que pueden presentar algunas diferencias respecto de las ya estudiadas, al menos en aspectos muy puntuales.

Un árbol es una estructura jerárquica de un conjunto de elementos u objetos llamados nodos o vértices, uno de los cuales se distingue como raíz, que establece una relación de “parentesco” entre estos elementos, (que está dada por la disposición de las aristas) y que impone el uso de términos como *padre*, *hijo*, *hermano*, *antecesor*, *sucesor*.

A continuación definimos la estructura de árbol con raíz de manera recursiva:

Definición 6.44

Formalmente, a un árbol con raíz lo podemos definir de manera recursiva como sigue:

- i) Si $\emptyset(V) = \emptyset$, entonces este es el árbol vacío (no posee nodos).
- ii) Un nodo es en sí mismo un árbol. Ese nodo es la raíz de dicho árbol.
- iii) Sea n un nodo y $T_1, T_2, \dots, T_k$ árboles cuyos conjuntos de nodos son disjuntos y con raíces $n_1, n_2, \dots, n_k$, respectivamente. Podemos construir un nuevo árbol con raíz, definiendo a n como el padre de los nodos $n_1, n_2, \dots, n_k$. En el árbol construido de esta manera, n es la raíz y $T_1, T_2, \dots, T_k$ son los subárboles de la raíz. Los nodos $n_1, n_2, \dots, n_k$ reciben el nombre de hijos del nodo n .

Observaciones:

1. Cada nodo de un árbol es la raíz de un subárbol contenido en él. Al árbol se accede siempre por la raíz.

2. En un árbol con raíz, cada nodo diferente de la raíz tiene uno y sólo un padre.

Introducimos la terminología necesaria para continuar con el estudio del tema.

Un nodo X es *descendiente directo* del nodo Y , si X es hijo de Y .

Un nodo Z es *antecesor directo* del nodo Y , si Z es padre de Y .

Todos los nodos que son hijos de un mismo padre, son *hermanos*.

El *grado positivo* de un nodo es el número de descendientes directos que tiene.

El *nivel* de un nodo esta dado por el número de arcos que deben ser recorridos para llegar a él desde el nodo raíz. Así, el nivel del nodo raíz es cero, mientras que el nivel de cualquier otro nodo es uno más que el nivel de su padre.

La *altura* de un árbol es el valor máximo entre todos los niveles alcanzados por los nodos. La altura de un árbol será indicada por la letra "h".

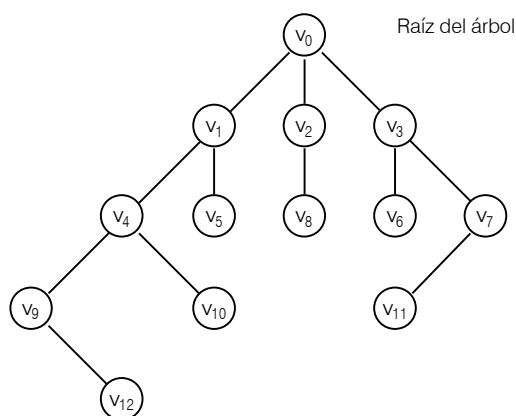
En la Figura 6.9 ii) la raíz v_0 es el padre de v_1 y v_2 ; v_3 , v_4 y v_5 son vértices denominados hojas o vástagos. Como v_4 y v_5 son hijos del mismo padre v_2 , se denominan hermanos mientras que v_3 es hijo de v_1 y no tiene hermanos.

Ejemplo 26

En la Figura 6.16 tenemos que el nodo v_0 es padre de v_1 , v_2 , v_3 ; el nodo v_{12} es descendiente de v_9 , v_4 , v_1 , v_0 ; el nodo v_{12} es descendiente directo (hijo) de v_9 ; los nodos del nivel 1 son v_1 , v_2 , v_3 ; los nodos del nivel 2 son v_4 , v_5 , v_8 , v_6 , y v_7 .

Como el nivel de un nodo es la longitud del único camino entre la raíz y dicho nodo tenemos que los nodos v_9 , v_{10} y v_{11} están en el nivel 3. En el ejemplo, el nodo v_{12} está en el nivel 4 que es el mayor nivel alcanzado por los nodos, por lo que la altura del árbol es $h=4$.

Figura 6.16

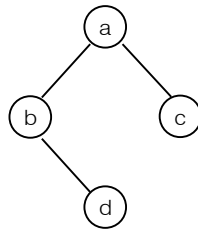


Ejemplo 27

En la Figura 6.17, d es *descendiente* directo de b ; b y c son *hijos* o descendientes direc-

tos de a . El nodo a es *padre* de b y c ; b es *antecesor* directo o padre de d ; b y c son *hermanos*, d no tiene hermanos. La raíz a está en el nivel 0, b y c se encuentran en el nivel 1, mientras que d está en el nivel 2. Los vértices c y d son hojas mientras que los nodos a y b son nodos internos o de ramificación.

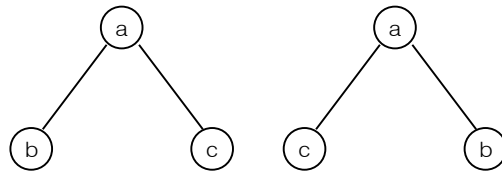
Figura 6.17



6.9.1 Los árboles como estructuras ordenadas

A menudo los hijos de un nodo se consideran ordenados de izquierda a derecha. De esta forma, los árboles de la siguiente figura son distintos, porque aunque tienen los mismos nodos (a, b, c) y los hijos de cada nodo son los mismos, los hijos del nodo " a " aparecen en distinto orden.

Figura 6.18



Un problema interesante que nos planteamos en estas estructuras de árbol donde importa el orden de los nodos es el *recorrido* de los mismos. Este consiste básicamente en obtener un ordenamiento de todos los nodos del árbol. En lo sucesivo, mostraremos los ordenamientos de los nodos de un árbol como una lista de nodos. Por ejemplo las listas " a, b, d, c " y " d, b, c, a " muestran dos ordenamientos distintos de los nodos del árbol de la figura 6.17. Asimismo, cada uno de estos ordenamientos es un recorrido distinto de dicho árbol.

Dado que los árboles son estructuras jerárquicas (no lineales) podemos pensar en distintos recorridos (empezar por las hojas, empezar por la raíz, etc.). Sin embargo, existen determinadas formas de recorrer árboles que suelen ser de utilidad en una gran cantidad de problemas. A continuación, analizaremos tres métodos muy conocidos en el estudio de las estructuras de datos para ordenar en forma sistemática los vértices de un árbol:

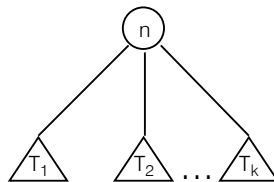
Definición 6.45

Las tres formas más importantes de recorrer un árbol se denominan *preorden* (u orden previo), *postorden* (u orden posterior) e *inorden* (u orden simétrico). Tales ordenamientos se definen *recursivamente* como sigue:

i) Si T contiene un solo nodo, entonces ese nodo constituye el listado de los nodos de T en los órdenes previo, simétrico y posterior.

ii) Si T contiene más de un nodo, cuya raíz es n y sus subárboles $T_1, T_2, \dots, T_k$, como se representa en la siguiente figura:

Figura 6.19



a) El listado en preorden de los nodos de T está formado por la raíz “ n ”, seguida de los nodos de T_1 recorridos en orden previo, luego por los nodos de T_2 recorridos en orden previo y así sucesivamente, finalizando con los nodos de T_k recorridos en orden previo.

b) El listado en inorden de los nodos de T está constituido por los nodos de T_1 recorridos en inorden, seguidos de la raíz “ n ” y luego por los nodos de T_2 recorridos en inorden, de T_3 recorridos en inorden, y así sucesivamente, finalizando con los nodos de T_k recorridos en inorden.

c) El listado en postorden de los nodos de T está formado por los nodos de T_1 recorridos en postorden, luego los nodos de T_2 recorridos en postorden y así sucesivamente hasta los nodos de T_k recorridos en postorden, finalizando con la raíz “ n ”.

Regla práctica:

Para el preorden se lista un nodo la primera vez que se pasa por él.

En el caso del postorden, se lista un nodo la última vez que se pasa por él.

Para el orden simétrico o inorden, se lista un nodo hoja la primera vez que se pasa por ella, y un nodo interno la segunda vez que se pasa por él.

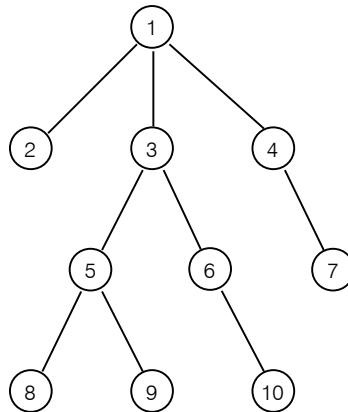
Ejemplo 28

Los listados en preorden, inorden y postorden de los nodos del árbol de la figura 6.20 son:

Listado en preorden: 1, 2, 3, 5, 8, 9, 6, 10, 4, 7.

Listado en inorden: 2, 1, 8, 5, 9, 3, 10, 6, 7, 4.
 Listado en postorden: 2, 8, 9, 5, 10, 6, 3, 7, 4, 1.

Figura 6.20



Ejemplo 29

Si recorremos el árbol de la Figura 6.17 obtenemos los siguientes listados:

Listado en preorden: a, b, d, c.

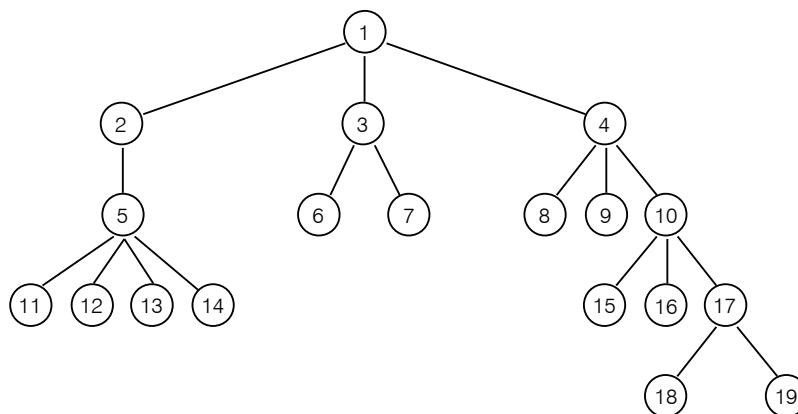
Listado en inorden: d, b, a, c.

Listado en postorden: d, b, c, a.

Ejemplo 30

Dado el siguiente árbol, listaremos sus nodos en preorden, inorden y postorden.

Figura 6.21



Listado en preorden: 1, 2, 5, 11, 12, 13, 14, 3, 6, 7, 4, 8, 9, 10, 15, 16, 17, 18, 19.

Listado en inorden: 11, 5, 12, 13, 14, 2, 1, 6, 3, 7, 8, 4, 9, 15, 10, 16, 18, 17, 19.

Listado en postorden: 11, 12, 13, 14, 5, 2, 6, 7, 3, 8, 9, 15, 16, 18, 19, 17, 10, 4, 1.

6.9.2 Árboles binarios

Los árboles representan una de las estructuras de datos más importantes en computación. Tienen múltiples aplicaciones. Una forma de utilizarlos es para representar una estructura en la cual es posible tomar decisiones con dos opciones como máximo. En un árbol binario cada nodo tiene a lo sumo dos subárboles, uno izquierdo y otro derecho. Cuando existe un solo subárbol debemos distinguir si se trata del izquierdo o del derecho.

Podemos definir formalmente un árbol binario, en forma recursiva, de la siguiente manera:

Definición 6.46

El árbol vacío es un árbol binario; un árbol de un sólo nodo es un árbol binario; dado un nodo "n" y dos árboles binarios B_1 y B_2 se puede construir el árbol binario B tomando a "n" como raíz, a B_1 como el hijo izquierdo de "n" y a B_2 como el hijo derecho de "n".

Insistimos que en un árbol binario, cada nodo tiene a lo sumo dos subárboles, y siempre se distinguen entre el subárbol izquierdo y el subárbol derecho (es un caso particular de árbol ordenado).

Si se adopta la convención de que los hijos izquierdos se dibujan a la izquierda de su padre y los hijos derechos hacia la derecha, las Figuras 6.22 a) y b) representan dos *árboles binarios distintos*, aunque ambos se representan como el mismo árbol con raíz de la Figura 6.23. Los árboles binarios distintos tienen estructuras y/o relaciones de parentesco diferentes.

La Figura 6.24 muestra *árboles binarios similares*. En éstos, sus estructuras son idénticas, pero la información contenida en sus nodos difiere entre sí.

Por último, la Figura 6.25 nos muestra *árboles binarios equivalentes*. Éstos tienen la misma estructura y la misma información en sus nodos.

Figura 6.22

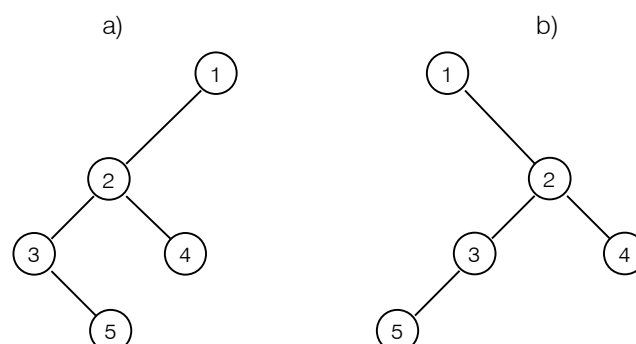


Figura 6.23

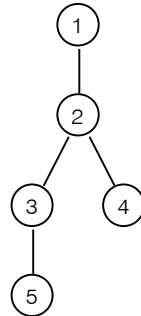


Figura 6.24

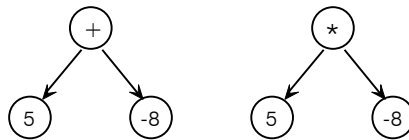
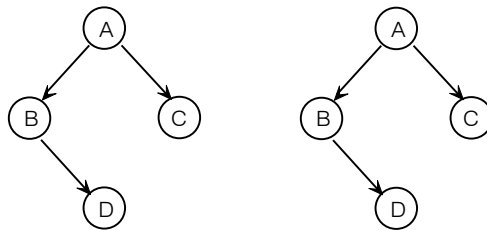


Figura 6.25

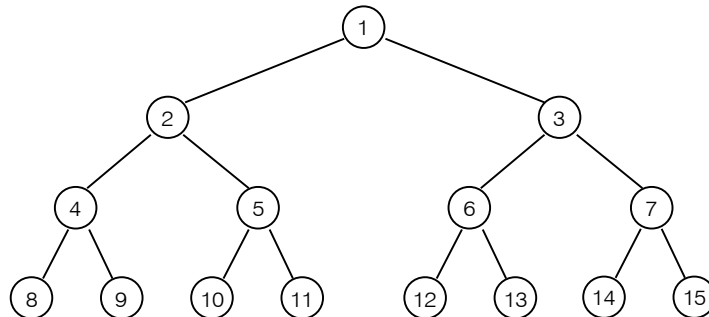


Definición 6.47

Se define un *árbol* (con raíz) *binario completo de altura h* a aquel en el que todos sus nodos de ramificación tienen grado positivo igual a dos y todas sus hojas se encuentran en el mismo nivel h .

En los árboles binarios completos, cada nodo, excepto las hojas, tiene dos hijos: un subárbol izquierdo y un subárbol derecho.

La siguiente figura representa un árbol binario completo de altura $h = 3$.



Observemos que en el nivel 0 se tiene 1 nodo; en el nivel 1, $2^1 = 2$ nodos; en el nivel 2, $2^2 = 4$ nodos y por último en el nivel 3 se tienen $2^3 = 8$ nodos.

Así, comenzando a contar los nodos desde la raíz y siguiendo con los nodos de cada nivel superior tendremos que el número de nodos del árbol es la raíz más la suma de las "3" primeras potencias de 2 (siendo 3 la altura del árbol). Esto es:

$$1 + 2 + 4 + 8 = 15 \text{ nodos.}$$

Este ejemplo induce la siguiente generalización:

Si h representa la altura de un árbol binario completo, el número de nodos de cada nivel natural sucesivo está dado por los términos de la sucesión geométrica 2, 4, 8, ..., 2^h .

Luego el número de nodos del árbol es $1 + 2 + 4 + 8 + \dots + 2^h$ que es la suma de los primeros términos de una sucesión geométrica de razón 2. Evaluando esta suma tendremos que, en un árbol binario completo de altura " h ", el número de nodos es:

$$1 + 2 + 4 + 8 + \dots + 2^h = 2^{h+1} - 1.$$

Dejamos a cargo del lector verificar, utilizando la inducción, la veracidad de la identidad anterior.

6.9.3 Recorrido de árboles binarios

Al igual que para el recorrido de árboles, pueden utilizarse tres maneras distintas para recorrer un árbol binario: preorden, inorden y postorden. Estos tres métodos, dados recursivamente, se reescriben para árboles binarios, como se indica a continuación:

Recorrido en preorden

Visitar la raíz
Recorrer el subárbol izquierdo (left) en preorden
Recorrer el subárbol derecho (right) en preorden

Recorrido en inorden

Recorrer el subárbol izquierdo (left) en inorden
Visitar la raíz
Recorrer el subárbol derecho (right) en inorden

Recorrido en postorden

Recorrer el subárbol izquierdo (left) en postorden
Recorrer el subárbol derecho (right) en postorden
Visitar la raíz

Ejemplo 31

Tomando la Figura 6.22 a) obtenemos los siguientes recorridos para el árbol binario:

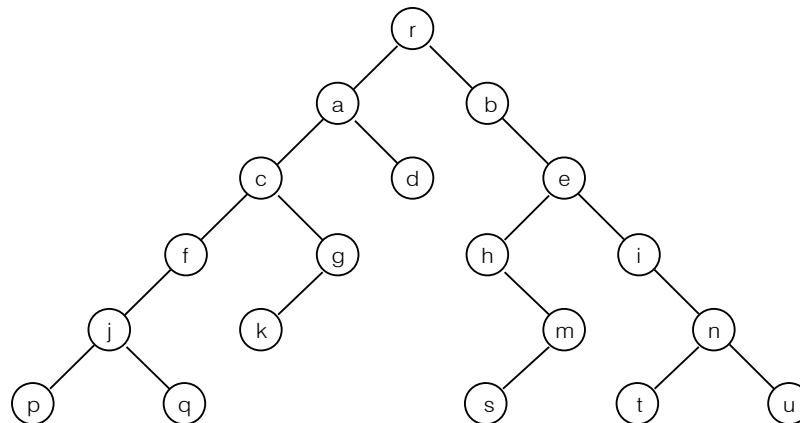
Preorden: 1 2 3 5 4

Inorden: 3 5 2 4 1

Postorden: 5 3 4 2 1

Observación:

Veamos la Figura 6.26



Si aplicamos el recorrido en inorden al *árbol binario* que se muestra en la Figura 6.26 veremos que la lista en inorden es:

p, j, q, f, c, k, g, a, d, r, b, h, s, m, e, i, t, n, u.

Si pensamos el árbol precedente como un *árbol con raíz* y aplicamos la definición para la lectura en inorden de sus nodos tenemos:

p, j, q, f, c, k, g, a, d, r, s, m, h, e, t, n, u, i, b.

Será preciso tener en cuenta si el árbol dirigido dado, se piensa como *árbol binario* o simplemente como *árbol con raíz*, porque en el caso de recorridos de los nodos en inorden, se producen lecturas diferentes.

La estructura en árbol binario puede representarse gráficamente de varias maneras. Hasta ahora nos hemos dedicado a la representación de árboles por medio de grafos.

Otra forma habitual de representar un árbol es utilizando una notación común en el área de programación, tal como *una notación secuencial* mediante *paréntesis anidados*.

Definimos la *notación prefija* como:

(R (Subárbol Izquierdo, Subárbol Derecho))

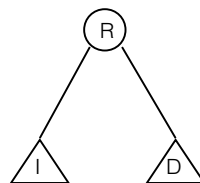
Mientras que la *notación postfija* se define como:

((Subárbol Izquierdo, Subárbol Derecho) R)

donde R es la raíz; Subárbol Izquierdo hace referencia al hijo izquierdo de R y Subárbol Derecho hace referencia al hijo derecho de R. La ausencia de alguno de los subárboles será indicada con λ , el árbol vacío.

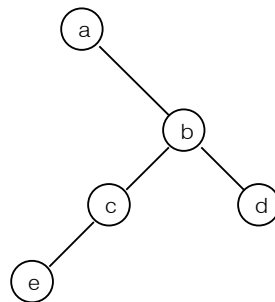
La siguiente figura muestra un árbol binario B, donde R, es la raíz de B, I el hijo izquierdo de R y D el hijo derecho de R. Su notación empleando paréntesis anidados es $(R(I, D))$ o $((I, D)R)$.

Figura 6.27



Ejemplo 32

Consideremos el siguiente árbol binario:



La notación prefija utilizando paréntesis anidados es la siguiente:

$(a(\lambda, b(c(e, \lambda), d)))$

La notación postfija utilizando paréntesis anidados es la siguiente:

$((\lambda, ((e, \lambda) c, d) b) a)$

Si el árbol binario dado se recorre en *preorden*, estaríamos listando los nodos en la secuencia “a b c e d”, que no es más que la expresión $(a(\lambda, b(c(e, \lambda), d)))$, sin paréntesis ni comas, ni subárboles vacíos.

El recorrido en *postorden* del mismo árbol genera la siguiente lista de nodos: “e c d b a”, que se puede obtener de la expresión $((\lambda, ((e, \lambda) c, d) b) a)$, eliminando paréntesis, comas y árboles vacíos.

6.9.4 Los árboles como estructuras etiquetadas

Muchos problemas de las especialidades de la Informática pueden representarse usando la estructura de árbol, asociando un valor o etiqueta a cada uno de los nodos. La etiqueta de

un nodo no será el nombre del nodo, sino un valor “almacenado” (guardado) en él.

Una de las aplicaciones para utilizar la estructura de árbol etiquetado es la representación de expresiones aritméticas. Veamos como hacerlo.

Para ello definiremos, en particular, como construir un árbol etiquetado para expresiones aritméticas definidas sobre los enteros y en especial, las que corresponden a operadores binarios.

Definición 6.48

Un árbol etiquetado para una expresión aritmética se construye de acuerdo a las siguientes reglas:

- i) El árbol correspondiente a un solo símbolo o variable numérica es un nodo que tiene a ese símbolo como etiqueta.
- ii) El árbol de una fórmula que consiste en un operador binario y sus dos argumentos, por ejemplo “ $a \bullet b$ ”, se forma construyendo un nodo y etiquetándolo con el operador “ $\bullet$ ”; se construye el árbol para la expresión “ a ” y se lo define como un subárbol izquierdo de “ $\bullet$ ” y, de manera similar, se construye el árbol para “ b ” y se lo toma como subárbol derecho de “ $\bullet$ ”.

Ejemplo 33

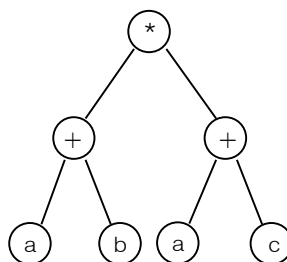
Para obtener el árbol etiquetado que represente la expresión aritmética:

$$((a + b) * (a + c))$$

seguimos el procedimiento descrito anteriormente.

En primer lugar notemos que $*$ corresponde a la raíz del árbol, donde sus subárboles izquierdo y derecho son $(a + b)$ y $(a + c)$, respectivamente. Las raíces de estos subárboles corresponden al operador $+$, y los subárboles izquierdo y derecho de cada uno de ellos son, respectivamente, a y b para el subárbol izquierdo, y a y c para el subárbol derecho.

Figura 6.28



Ejemplo 34

Obtendremos el árbol etiquetado que representa la expresión aritmética:

$$(a - b * c) + d - e / f$$

Si introducimos paréntesis para separar en términos se tiene:

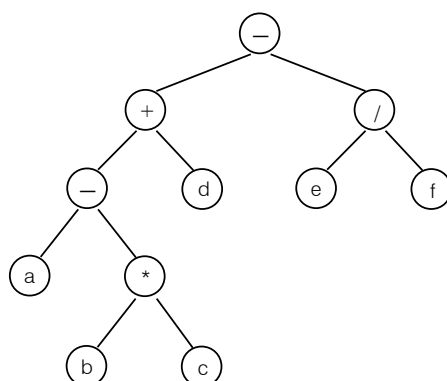
$$(a - b * c) + d - (e / f)$$

Como los operadores adición(+) y sustracción(-) tienen igual prioridad, la evaluación de la expresión se realiza de izquierda a derecha.

Entonces, en primer lugar calculamos $((a - b * c) + d)$ y luego, al resultado de la misma, le restamos (e / f) . Por lo tanto, la raíz del árbol está etiquetada por el operador "-".

Así el árbol binario que representa dicha expresión es el correspondiente a la siguiente figura:

Figura 6.29



En el caso de un árbol que representa una expresión aritmética, el listado en *preorden* de las etiquetas de los nodos da lo que se conoce como *forma prefija de la expresión* o *forma polaca*, en la cual un operador precede a sus operandos izquierdo y derecho. La forma polaca de una expresión algebraica es interesante ya que representa la expresión de manera no ambigua sin necesidad del uso de paréntesis.

El listado en *postorden* de las etiquetas de los nodos de un árbol que representa una expresión aritmética da lo que se conoce como *representación postfija* o *sufija*, en la cual un operador está colocado después de sus argumentos. Como en el caso anterior, esta expresión no requiere de paréntesis.

El recorrido en *inorden* de un árbol que representa una expresión aritmética, da la *expresión infija* de la misma. En general, esta expresión es ambigua y para obtener una única representación necesita del uso de paréntesis.

Ejemplo 35

El listado en preorden del árbol de la Figura 6.28 es: $* + a b + a c$. Observa que la notación prefija no necesita de paréntesis.

La expresión sufija correspondiente a la misma figura es $ab + ac + *$ que se obtiene al utilizar el postorden para el ordenamiento de los nodos. Nuevamente vemos que los paréntesis son innecesarios.

Veamos que sucede si utilizamos el recorrido en inorden.

Ejemplo 36

El listado en inorden de las etiquetas de la Figura 6.28 es $a + b * a + c$. Esta expresión es ambigua, puede pensarse como $a + (b * a) + c$ o $(a + b) * (a + c)$ generando árboles distintos.

Los árboles que muestran la estructura operador-operando de una expresión se conocen a veces como árboles de sintaxis abstracta, debido a que muestran la estructura sintáctica de una expresión sin importar la notación en la cual se escribió originalmente la expresión.

Actividad 7

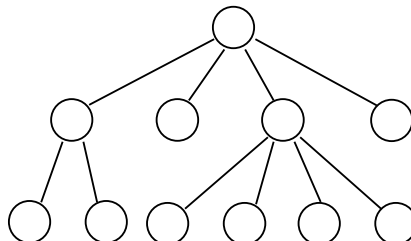
Problema N°1: Construye un árbol con raíz que satisfaga simultáneamente las siguientes relaciones:

- a) H, I, J son hijos de G
- b) D es el padre de F y E
- c) E es padre de G

Problema N°2: Lee el siguiente algoritmo:

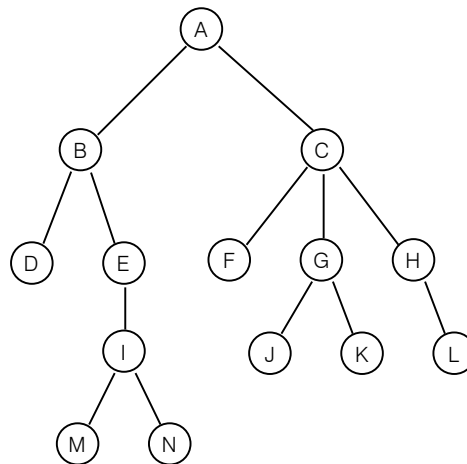
1. Asignar la etiqueta (o nombre) 0 a la raíz.
2. Asignar los enteros positivos 1, 2, 3, ..., m, a los vértices que están unidos a la raíz por una cadena de longitud 1 de izquierda a derecha. (Tales vértices se dice que están en el nivel 1)
3. Si (v es un vértice del nivel $n \geq 1$) y ($v_1, v_2, v_3, \dots, v_k$ los hijos de v de izquierda a derecha) entonces
 - 3.1. si a es la etiqueta asignada al vértice v entonces
 - 3.1.1. asignar $a.1, a.2, a.3, \dots, a.k$ a los hijos $v_1, v_2, v_3, \dots, v_k$ respectivamente.

Dado el siguiente árbol, coloca las etiquetas de los vértices según siguiendo el algoritmo.



Debido a que estas etiquetas se colocan como el orden alfabético de un diccionario, el orden se denomina *lexicográfico* o *de diccionario*.

Problema N°3: Considerando este árbol con raíz responde las siguientes preguntas:



- 3.1) ¿Qué nodos son hojas?
- 3.2) ¿Qué nodo es la raíz?
- 3.3) ¿Cuál es el padre de C?
- 3.4) ¿Qué nodos son los hijos de C?
- 3.5) ¿Cuáles son los hermanos de K?

Problema N°4: Lista los nodos del árbol con raíz dado en el problema anterior en:

- 4.1) preorden
- 4.2) inorden
- 4.3) postorden

Problema N°5: Convierte la expresión $((a + b) + c * (d + e) + f) * (g + h)$ en:

- 5.1) su forma prefija
- 5.2) su forma postfija

Problema N°6: Construye el árbol binario para cada una de las siguientes expresiones algebraicas:

- 6.1) $((2 * x) + (3 - (4 * x))) + (x - (3 * 11))$
- 6.2) $3 + 5 \bmod 7 * (-1 + 4)$
- 6.3) $(7 + (6 - 2)) - (x - (y - 4))$
- 6.4) $3 - (x + 6 * (2 - 3))$
- 6.5) $2 + 4 * 3 - 6 / (5 - 3)$

Problema N°7: Indica la notación prefija y postfija de cada una de las expresiones dadas en el Problema N°6.

Problema N°8: Dada la lista de números: 25, 17, 9, 20, 13, 30

8.1) Construye un árbol binario de raíz 25 que contenga los elementos de la lista dada, de manera tal que al recorrerlo en inorden obtengamos la lista de dichos elementos pero ordenados ascendentemente.

8.2) Resuelve el ítem 8.1) pero considerando que el recorrido en inorden deberá generar la lista de los elementos ordenados descendentemente.

Problema N°9: Dibuja los árboles binarios correspondientes a las siguientes expresiones, dadas mediante paréntesis anidados:

9.1) $(x(\lambda, y(z(w, \lambda), t)))$

9.2) $(x(\lambda, \lambda))$

9.3) $(1(2(3, \lambda), \lambda))$

9.4) $(a(\lambda, h))$

9.5) $(z(x(a, \lambda), y(b(c, y), \lambda)))$

9.6) $(((\lambda, 3) 2, ((7, 6) 5, \lambda) 4) 1)$

9.7) $((((3, \lambda) 2, (\lambda, 5) 4) 1)$

Problema N°10: Para cada una de las siguientes expresiones aritméticas dadas en notación postfija:

10.1) $a b c d + * / e -$

10.2) $a b c * * c d e + / -$

a) Dibuja el árbol binario que la representa.

b) Obtiene la notación prefija.

Problema N°11: En un árbol binario completo con 1.023 nodos,

11.1) ¿Cuál es la altura?

11.2) ¿Cuántos nodos están en el nivel 6?

11.3) ¿Cuántos nodos de ramificación hay?

Problema N°12: En un árbol binario de altura 7, ¿cuáles son el mínimo y el máximo número posible de nodos que puede tener?

6.10 Árboles de refutación de una fórmula de la LPC

En el Capítulo 1 hemos visto que las tablas de verdad constituyen un método para determinar la satisfacibilidad de una fórmula o conjunto de fórmulas de la LPC. En esta sección

presentaremos un método alternativo y más genérico, basado en la construcción de árboles, y que puede ser extendido para el cálculo de predicados.

Recordemos que denotamos con Form al conjunto de todas las fórmulas bien formadas (fbf) o simplemente fórmulas de la LPC.

Sea S un conjunto de fórmulas, $S \subseteq \text{Form}$ y ϕ una fórmula. Sabemos que bajo una cierta valuación v , algunas fórmulas de S pueden ser verdaderas y otras falsas y lo mismo ocurre para la fórmula ϕ . Hemos visto que una fórmula ϕ es consecuencia de un conjunto de premisas S si toda valuación que es un modelo para S es también un modelo para ϕ . Esto es, si $v(\theta) = 1$ para toda fórmula $\theta \in S$, entonces $v(\phi) = 1$.

En el Ejemplo 14 del Capítulo 1, hemos visto que $\phi = (p \rightarrow r)$ es consecuencia del conjunto de fórmulas $S = \{(p \rightarrow q), (q \rightarrow r)\}$. Esto implica que toda interpretación que hace verdaderas a las fórmulas de S , también hace verdadera a la fórmula ϕ . Una manera de demostrar esto es construyendo las tablas de verdad de las fórmulas que intervienen en la premisa y la de la conclusión. En el caso de existir una interpretación que haga simultáneamente verdaderas a estas fórmulas, podemos concluir que ϕ es consecuencia de S , es decir $S \models \phi$. Una manera alternativa de probar esta implicación lógica es aplicando en primer lugar el teorema de la deducción. Este teorema nos asegura que $\{(p \rightarrow q), (q \rightarrow r)\} \models (p \rightarrow r)$ si y sólo si $\{(p \rightarrow q), (q \rightarrow r), p\} \models r$, es decir $S \cup \{p\} \models r$. Luego, construimos la tabla de verdad de la fórmula $((p \rightarrow q) \wedge (q \rightarrow r) \wedge p) \rightarrow r$, como se muestra a continuación:

Tabla 6.1

p	q	r	$p \rightarrow q$	$q \rightarrow r$	$(p \rightarrow q) \wedge (q \rightarrow r) \wedge p$	$((p \rightarrow q) \wedge (q \rightarrow r) \wedge p) \rightarrow r$
0	0	0	1	1	0	1
0	0	1	1	1	0	1
0	1	0	1	0	0	1
0	1	1	1	1	0	1
1	0	0	0	1	0	1
1	0	1	0	1	0	1
1	1	0	1	0	0	1
1	1	1	1	1	1	1

El único renglón que nos interesa es aquel en el que todas las premisas son verdaderas. El conjunto $S \cup \{p\}$ se dice satisfacible porque bajo la interpretación $I = \{p, q, r\}$ las fórmulas de $S \cup \{p\}$ son todas verdaderas. Como bajo esta interpretación I la conclusión también es verdadera, podemos concluir que $S \cup \{p\} \models r$, o de manera equivalente que $\{(p \rightarrow q), (q \rightarrow r)\} \models (p \rightarrow r)$.

En el Capítulo 5 hemos estudiado un álgebra de Boole muy particular con innumerables aplicaciones a las ciencias de la computación e información: el Álgebra de Boole Binaria o

Álgebra de Boole Bivaluada. Esta álgebra booleana es denotada por el quinteto $(\{0, 1\}, +, \cdot, 0, 1)$ o simplemente por el conjunto $B = \{0 \text{ (Falso, False); } 1 \text{ (Verdadero, True)}\}$. Las operaciones binarias “+” y “ $\cdot$ ” son representadas por los conectivos “ $\vee$ ” y “ $\wedge$ ”, respectivamente, y la operación unaria de complementación por el conectivo “ $\neg$ ”. Hemos visto que si interpretamos a las variables booleanas como números enteros con las operaciones y el orden habituales, entonces $B = \{0, 1\}$ es un álgebra de Boole con las operaciones:

$$x + y = \max \{x, y\}, \quad x \cdot y = \min \{x, y\}, \quad \text{y la complementación como } x' = 1 - x.$$

Esta álgebra booleana permite dar una definición alternativa del concepto de valuación dado en el Capítulo 1.

Definición 6.49

Una valuación booleana o simplemente valuación es cualquier función v definida en el conjunto Form con imágenes en el conjunto B de las variables booleanas, es decir $v: \text{Form} \rightarrow B$, que satisface las siguientes condiciones, siendo θ y ϕ fórmulas arbitrarias:

$$v(\neg\theta) = 1 - v(\theta)$$

$$v(\theta \wedge \phi) = \min\{v(\theta), v(\phi)\}$$

$$v(\theta \vee \phi) = \max\{v(\theta), v(\phi)\}$$

$$v(\theta \rightarrow \phi) = \max\{1 - v(\theta), v(\phi)\}$$

Ejemplo 37

Aquí retomamos el ejemplo 7 del Capítulo 1 para analizar si las fórmulas dadas son satisfacibles o no, utilizando la nueva definición de valuación.

a) Sea $\text{Var} = \{p_1, p_2, p_3\}$ y la fórmula $\theta = (p_1 \rightarrow (p_2 \vee \neg p_3))$.

$v(\theta) = v(p_1 \rightarrow (p_2 \vee \neg p_3)) = \max\{1 - v(p_1), v(p_2 \vee \neg p_3)\} = \max\{1 - v(p_1), \max\{v(p_2), v(\neg p_3)\}\} = \max\{1 - v(p_1), \max\{v(p_2), 1 - v(p_3)\}\}$. Para completar la valuación de la fórmula, necesitamos conocer la valuación de las fórmulas atómicas que la componen. Por ejemplo, siendo $v(p_1) = 0$, $v(p_2) = 1$ y $v(p_3) = 1$ obtenemos que $v(\theta) = 1$. Entonces, bajo esta interpretación la fórmula θ es satisfacible.

b) Sea ahora la fórmula $\phi = \neg(p_1 \vee \neg p_3)$.

Bajo la misma interpretación para las variables proposicionales, esto es $v(p_1) = 0$, $v(p_2) = 1$ y $v(p_3) = 1$, se verifica que $v(\phi) = 1 - v(p_1 \vee \neg p_3) = 1 - \max\{v(p_1), 1 - v(p_3)\} = 1 - \max\{0, (1 - 1)\} = 1 - 0 = 1$, con lo cual la fórmula ϕ es satisfacible.

Entonces, es posible concluir que si $S = \{\theta, \phi\}$, existe una valuación $v(p_1) = 0$, $v(p_2) = 1$ y $v(p_3) = 1$ que hace verdadera a todas las fórmulas de S , por lo que podemos afirmar que el conjunto de fórmulas S es satisfacible.

A continuación describiremos otro procedimiento basado en la construcción de árboles para determinar si un conjunto finito y no vacío de fórmulas es o no satisfacible, el cual puede generalizarse para el cálculo de predicados, lo que no ocurre con las tablas de verdad.

El siguiente lema resume los valores de verdad que asumen las fórmulas atendiendo a cómo están construidas.

Lema 6.1

Sean θ, φ fórmulas arbitrarias y v una valuación. Entonces se verifican las siguientes reglas:

- $(R\neg)$ v satisface a $\neg\neg\theta$ si y sólo si v satisface a θ .
- $(R\vee)$ v satisface a $(\theta \vee \varphi)$ si y sólo si v satisface a θ o v satisface a φ .
- $(R\neg\vee)$ v satisface a $\neg(\theta \vee \varphi)$ si y sólo si v satisface a $\neg\theta$ y v satisface a $\neg\varphi$.
- $(R\wedge)$ v satisface a $(\theta \wedge \varphi)$ si y sólo si v satisface a θ y v satisface a φ .
- $(R\neg\wedge)$ v satisface a $\neg(\theta \wedge \varphi)$ si y sólo si v satisface a $\neg\theta$ o v satisface a $\neg\varphi$.
- $(R\rightarrow)$ v satisface a $(\theta \rightarrow \varphi)$ si y sólo si v satisface a $\neg\theta$ o v satisface a φ .
- $(R\neg\rightarrow)$ v satisface a $\neg(\theta \rightarrow \varphi)$ si y sólo si v satisface a θ y v satisface a $\neg\varphi$.

Las reglas de comportamiento de los conectivos con respecto a la satisfacibilidad se pueden esquematizar así:

Tipo I	
$R\neg$	$\frac{\neg\neg\theta}{\theta}$
Tipo C	Tipo D
$R\neg\vee$	$R\vee$
$\frac{\neg(\theta \vee \varphi)}{\neg\theta, \neg\varphi}$	$\frac{\theta \vee \varphi}{\theta \mid \varphi}$
$R\wedge$	$R\neg\wedge$
$\frac{\theta \wedge \varphi}{\theta, \varphi}$	$\frac{\neg(\theta \wedge \varphi)}{\neg\theta \mid \neg\varphi}$
$R\neg\rightarrow$	$R\rightarrow$
$\frac{\neg(\theta \rightarrow \varphi)}{\theta, \neg\varphi}$	$\frac{\theta \rightarrow \varphi}{\neg\theta \mid \varphi}$

Las fórmulas que figuran encima de la línea horizontal se llaman premisas y las que figuran debajo se llaman conclusiones de la regla.

Obtenemos tres tipos de reglas: las de *Tipo I* que tienen una conclusión única; las de *Tipo C* con dos conclusiones separadas por comas y las de *Tipo D* que tienen dos conclusiones separadas por una barra vertical. Las reglas de tipo C significan que una valuación satisface a la premisa si y sólo si satisface a las dos conclusiones, y las de tipo D significan que una valuación satisface a la premisa si y sólo si satisface por los menos a una de las dos conclusiones.

Lema 6.2

Si θ es una fórmula, entonces uno y sólo uno de los siguientes enunciados es verdadero:

- i) θ es una variable proposicional,
- ii) θ es la negación de una variable proposicional,
- iii) θ es de la forma de la premisa de una y sólo una de las reglas $R\neg$, $R\vee$, $R\neg\vee$, $R\wedge$, $R\neg\wedge$, $R\rightarrow$, $R\neg\rightarrow$.

A continuación usaremos las reglas antes enunciadas para construir un árbol que nos permitirá decidir sobre la satisfacibilidad de una fórmula o conjunto de fórmulas.

Definición 6.50

Sea A un árbol cuyos nodos son fórmulas. Un árbol A' se dice que es una *extensión inmediata* de A si A' se obtiene por aplicación de una de las reglas $R\neg$, $R\vee$, $R\neg\vee$, $R\wedge$, $R\neg\wedge$, $R\rightarrow$, $R\neg\rightarrow$ a una fórmula θ que figure en A del modo siguiente:

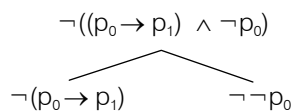
- a) Si la fórmula θ es del tipo de la premisa de la regla de tipo I, entonces A' se obtiene agregando un nuevo nodo terminal con la única conclusión de θ como único sucesor del nodo terminal de la rama de A donde figura θ .
- b) Si la fórmula θ es del tipo de la premisa de una regla de tipo C, entonces A' se obtiene agregando un nuevo nodo terminal con una de las conclusiones de θ como único sucesor del nodo terminal de una rama de A donde figura θ .
- c) Si la fórmula θ es del tipo de la premisa de una regla de tipo D, entonces A' se obtiene agregando dos nuevos nodos terminales, correspondientes a cada una de las conclusiones de θ , como sucesores del nodo terminal de una rama de A en la que figura θ .

Observación:

Teniendo en cuenta que los nodos de los árboles se corresponden con fórmulas del Cálculo Proposicional, y que por tal motivo pueden ser expresiones con un gran número de variables proposicionales y conectivos, omitiremos la representación gráfica de los nodos mediante circunferencias.

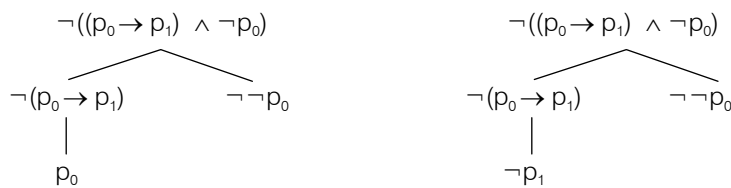
Ejemplo 38

Sea la fórmula $\theta = \neg((p_0 \rightarrow p_1) \wedge \neg p_0)$. Consideremos a θ como la raíz de un árbol A cuyo único nodo es la raíz. Como θ es de la forma de la premisa de la regla $R\neg\wedge$ (Tipo D), aplicaremos c) para obtener una extensión inmediata de A . Luego agregamos dos nodos terminales para tener en cuenta ambas posibilidades.

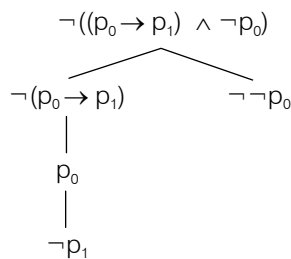


Podemos observar que una valuación que satisfaga la fórmula de la raíz del árbol, esto es $\neg((p_0 \rightarrow p_1) \wedge \neg p_0)$, va a satisfacer a por lo menos una de las dos conclusiones, es decir a $\neg(p_0 \rightarrow p_1)$ o a $\neg\neg p_0$.

Si continuamos con la rama izquierda, como $\neg(p_0 \rightarrow p_1)$ es de la forma de la premisa de la regla $R\rightarrow$ (Tipo C), aplicando b) obtenemos dos posibles extensiones inmediatas de A:

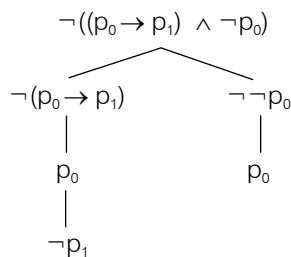


Si tomamos el árbol de la izquierda, podemos aplicar nuevamente b) y así obtener una nueva extensión inmediata de A:



Aquí podemos observar que una valuación que satisfaga la fórmula $\neg(p_0 \rightarrow p_1)$, debe satisfacer a las dos conclusiones de esta regla, esto es p_0 y $\neg p_1$.

Por último, como $\neg\neg p_0$ es de la forma de la premisa de la regla $R\neg$ (Tipo I), aplicando a) obtenemos otra extensión inmediata de A.



Como las fórmulas obtenidas de la aplicación sucesiva de c), b), b) y a) son variables proposicionales o negaciones de variables proposicionales, ya no es posible obtener otras extensiones inmediatas de A.

El árbol obtenido se denomina *árbol de refutación de la fórmula θ* .

Definición 6.51

Un árbol de fórmulas es un *árbol de refutación de una fórmula* θ si y sólo si se obtiene realizando un número finito de extensiones inmediatas a partir del árbol cuyo único nodo es la fórmula θ .

Definición 6.52

Un árbol de refutación de un conjunto finito S de fórmulas es un árbol de refutación de la conjunción de las fórmulas de S .

Ejemplo 39

Veamos como se construye un árbol de refutación del conjunto de fórmulas:

$$S = \{\neg(p_0 \wedge p_1), (p_0 \rightarrow p_2), (p_1 \vee \neg p_2)\}.$$

La raíz del árbol es la conjunción de las fórmulas de S . Esto es:

$$(\neg(p_0 \wedge p_1) \wedge (p_0 \rightarrow p_2)) \wedge (p_1 \vee \neg p_2)$$

Luego, como esta fórmula es de la forma de la premisa de la regla $R_{\wedge}$, una valuación v debe satisfacer a $(\neg(p_0 \wedge p_1) \wedge (p_0 \rightarrow p_2))$ y a $(p_1 \vee \neg p_2)$. Luego un árbol que es una extensión inmediata del anterior es:

$$\begin{array}{c} (\neg(p_0 \wedge p_1) \wedge (p_0 \rightarrow p_2)) \wedge (p_1 \vee \neg p_2) \\ | \\ (\neg(p_0 \wedge p_1) \wedge (p_0 \rightarrow p_2)) \\ | \\ (p_1 \vee \neg p_2) \end{array}$$

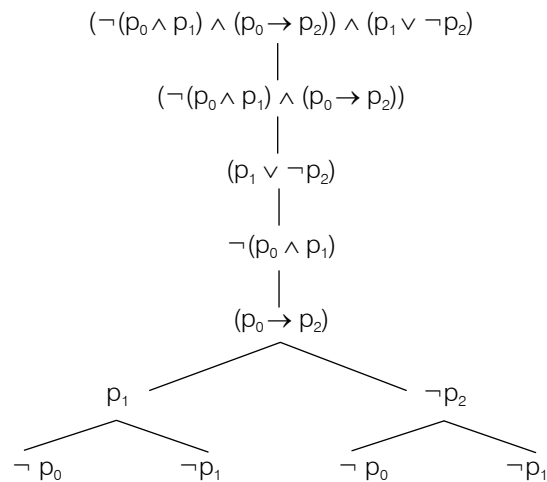
Si aplicamos de manera sucesiva el inciso b) de la definición 6.50 a la fórmula $(\neg(p_0 \wedge p_1) \wedge (p_0 \rightarrow p_2))$, debemos agregar de manera consecutiva dos nuevos nodos correspondientes a las fórmulas $\neg(p_0 \wedge p_1)$ y $(p_0 \rightarrow p_2)$ a continuación del nodo terminal $(p_1 \vee \neg p_2)$ que figura debajo de la fórmula $(\neg(p_0 \wedge p_1) \wedge (p_0 \rightarrow p_2))$. Así, obtenemos:

$$\begin{array}{c} (\neg(p_0 \wedge p_1) \wedge (p_0 \rightarrow p_2)) \wedge (p_1 \vee \neg p_2) \\ | \\ (\neg(p_0 \wedge p_1) \wedge (p_0 \rightarrow p_2)) \\ | \\ (p_1 \vee \neg p_2) \\ | \\ \neg(p_0 \wedge p_1) \\ | \\ (p_0 \rightarrow p_2) \end{array}$$

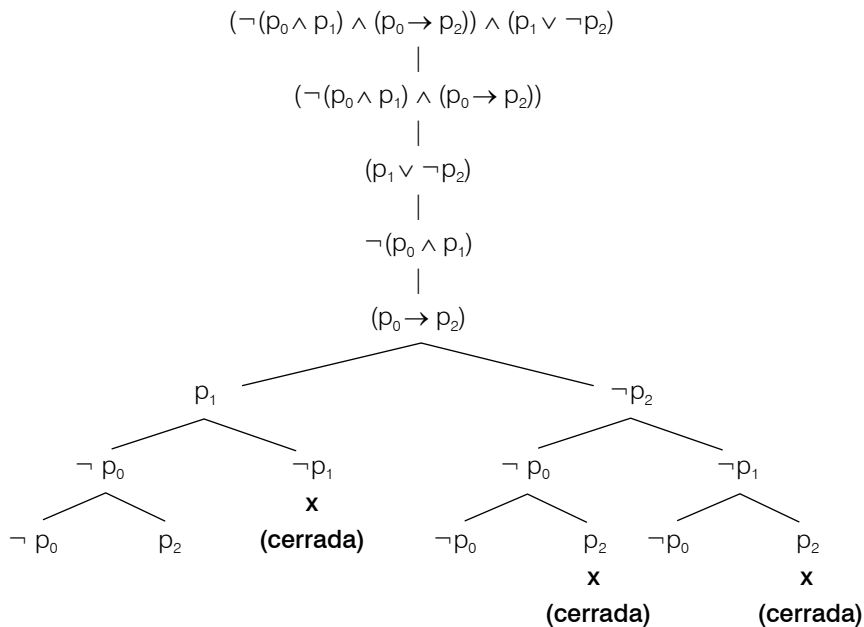
Observemos que las fórmulas $(p_1 \vee \neg p_2)$, $\neg(p_0 \wedge p_1)$ y $(p_0 \rightarrow p_2)$ son de la forma de las

premisas de las reglas de Tipo D. Por lo tanto, para cada una de ellas, una valuación que la satisfaga deberá satisfacer a por lo menos una de las dos conclusiones de esa premisa.

Si aplicamos c) a las fórmulas $(p_1 \vee \neg p_2)$ y $\neg(p_0 \wedge p_1)$, obtenemos:



Observemos que en una de las ramas de la izquierda del árbol figuran las fórmulas p_1 y $\neg p_1$. Como no puede existir una valuación que las satisfaga simultáneamente, el camino que contiene a ambas debe cerrarse, lo cual indicaremos con una "x" debajo del nodo terminal $\neg p_1$. Luego, es posible aplicar c) a la fórmula $(p_0 \rightarrow p_2)$, donde desde los restantes nodos terminales del árbol obtenido anteriormente se agregan dos nuevos nodos terminales correspondientes a $\neg p_0$ y p_2 .



En dos de las ramas de la derecha del árbol figuran las fórmulas p_2 y su negación, $\neg p_2$, por lo que dichas ramas deben cerrarse.

Lo analizado en el ejemplo previo da lugar a la siguiente definición:

Definición 6.53

Una rama de un árbol de fórmulas se dice cerrada si contiene a la vez una fórmula y su negación. En caso contrario, se dice abierta.

Definición 6.54

Un árbol de fórmulas se dice cerrado si todas sus ramas son cerradas.

Observación:

De acuerdo a la definición 6.53, en cada rama de un árbol cerrado aparece una fórmula y su negación, con lo cual no existen valuaciones que satisfagan a la conjunción de las fórmulas que conforman estas ramas. Como la raíz del árbol es parte del conjunto de fórmulas que se debe satisfacer para cada una de las ramas, podemos asegurar que no existen valuaciones que satisfagan a la fórmula que corresponde a la raíz. En consecuencia dicha fórmula es insatisfacible, es decir es una contradicción.

Definición 6.55

Un conjunto S de fórmulas es saturado si y sólo si satisface las siguientes condiciones:

- i) Una fórmula y su negación no pueden estar simultáneamente en S .
- ii) Si una fórmula de tipo C está en S , entonces sus dos conclusiones están en S .
- iii) Si una fórmula de tipo D está en S , al menos una de sus conclusiones está en S .

Ejemplo 40

a) Si $S = \{(p_0 \rightarrow p_2), \neg p_2, \neg p_1, p_2\}$ entonces S no es saturado. La condición (i) de la definición no se cumple porque p_2 y $\neg p_2$ están en S .

b) Si $S = \{\neg(p_0 \vee \neg p_1), (p_0 \rightarrow p_2), \neg p_0, p_1\}$, entonces por la definición anterior este conjunto es saturado. Podemos observar que la fórmula $\neg(p_0 \vee \neg p_1)$ es de la forma de la premisa de una regla del tipo C ($R \neg \vee$), y sus dos conclusiones $\neg p_0$ y p_1 están en S . De manera similar, la fórmula $(p_0 \rightarrow p_2)$ es de la forma de la premisa de una regla del tipo D ($R \rightarrow$), y por lo menos una de sus conclusiones, $\neg p_0$, está en S .

Definición 6.56

Una rama de un árbol de fórmulas se dice saturada si el conjunto formado por las fórmulas de sus nodos es saturado.

Ejemplo 41

El árbol del ejemplo 39 tiene tres ramas cerradas. Las restantes, que son abiertas, están

saturadas. Para cada una de estas ramas abiertas es posible encontrar una valuación que satisfaga al conjunto de fórmulas que la componen. Por ejemplo, para la primera rama del árbol, yendo de izquierda a derecha, la valuación $v(p_0) = 0$ y $v(p_1) = 1$ satisface a todas las fórmulas que la componen, en particular a la raíz del árbol. Por lo tanto, podemos concluir que la fórmula $(\neg(p_0 \wedge p_1) \wedge (p_0 \rightarrow p_2)) \wedge (p_1 \vee \neg p_2)$ es satisfacible o bien que el conjunto de fórmulas $S = \{\neg(p_0 \wedge p_1), (p_0 \rightarrow p_2), (p_1 \vee \neg p_2)\}$ es satisfacible.

Definición 6.57

Un árbol de refutación se dice completo si cada una de sus ramas es cerrada o saturada.

El árbol de refutación del ejemplo 39 es completo.

Observación:

De lo estudiado en el Capítulo 1 y lo analizado previamente podemos plantear las siguientes equivalencias:

θ es una tautología si y sólo si $\neg\theta$ es insatisfacible si y sólo si cualquier árbol de refutación completo de $\neg\theta$ es cerrado.

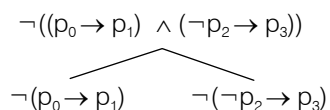
A continuación vamos a usar la construcción de árboles de refutación para decidir si una fórmula es una tautología, una contradicción o una contingencia.

Ejemplo 42

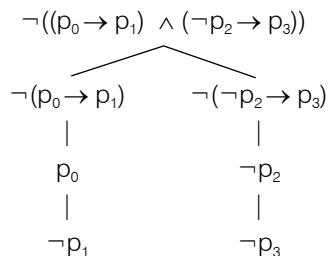
Sea $\theta = ((p_0 \rightarrow p_1) \wedge (\neg p_2 \rightarrow p_3))$. Veamos si esta fórmula es o no una tautología.

Sabemos que θ es una tautología si y sólo si $\neg\theta$ es insatisfacible. Si suponemos que $\neg\theta$ es satisfacible entonces debe existir una valuación v que la satisfaga. Veamos si esto es posible a través de la construcción de un árbol de refutación de $\neg\theta$.

Como $\neg\theta$ es de la forma de la premisa de la regla $R\neg\wedge$, aplicando dicha regla obtenemos el siguiente árbol:



Las fórmulas de los dos nodos terminales son de la forma de la premisa de la regla $R\neg\rightarrow$, con lo cual resulta:



Las dos ramas del árbol obtenido son saturadas, por lo que podemos decir que dicho árbol es completo.

Como el árbol posee por lo menos una rama abierta (en este caso todas son abiertas) podemos encontrar por lo menos una valuación que hace a $\neg\theta$ satisfacible. Sea v_1 una valuación tal que $v_1(p_0) = 1$ y $v_1(p_1) = 0$, entonces subiendo por la rama izquierda, v_1 satisface las fórmulas que figuran en la misma. Lo mismo sucede si tomamos una valuación v_2 tal que $v_2(p_3) = v_2(p_2) = 0$ y subimos por la rama derecha. Luego podemos concluir que $\neg\theta$ es satisfacible y por lo tanto θ no es una tautología.

Cabe aquí la siguiente pregunta: ¿Podemos afirmar que θ es entonces una contradicción?

Si así lo fuera, entonces $\neg\theta$ debe ser una tautología, es decir para toda valuación la fórmula debe ser satisfacible. Pero, observemos que si tomamos la valuación v_3 tal que $v_3(p_0) = 0$ y $v_3(p_2) = 1$ entonces $v_3(\neg\theta) = 0$, o lo que es equivalente $v_3(\theta) = 1 - v_3(\neg\theta) = 1$, con lo cual $\neg\theta$ y θ resultan ser ambas una contingencia.

Ejemplo 43

Veamos si $\phi = ((p_0 \wedge p_1) \rightarrow \neg(p_0 \vee \neg p_1))$ es o no una tautología.

Sabemos que ϕ es una tautología si y sólo si $\neg\phi$ es insatisfacible. Supongamos que $\neg\phi$ es satisfacible, entonces debe existir una valuación v que la satisfaga. Veamos si esto es posible.

Como $\neg\phi$ es de la forma de la premisa $R \rightarrow$, es aplicable esta regla, por lo que una valuación v debe satisfacer a $(p_0 \wedge p_1)$ y a $\neg\neg(p_0 \vee \neg p_1)$.

Dibujamos el árbol como:

$$\begin{array}{c} \neg((p_0 \wedge p_1) \rightarrow \neg(p_0 \vee \neg p_1)) \\ | \\ (p_0 \wedge p_1) \\ | \\ \neg\neg(p_0 \vee \neg p_1) \end{array}$$

Aplicamos la regla $R\neg\neg$ y agregamos como nuevo nodo terminal:

$$\begin{array}{c} \neg((p_0 \wedge p_1) \rightarrow \neg(p_0 \vee \neg p_1)) \\ | \\ (p_0 \wedge p_1) \\ | \\ \neg\neg(p_0 \vee \neg p_1) \\ | \\ (p_0 \vee \neg p_1) \end{array}$$

La regla $R\wedge$ es aplicable a $(p_0 \wedge p_1)$, con lo que obtenemos un nuevo árbol que es una extensión inmediata del anterior:

$$\begin{array}{c}
 \neg((p_0 \wedge p_1) \rightarrow \neg(p_0 \vee \neg p_1)) \\
 | \\
 (p_0 \wedge p_1) \\
 | \\
 \neg\neg(p_0 \vee \neg p_1) \\
 | \\
 (p_0 \vee \neg p_1) \\
 | \\
 p_0 \\
 | \\
 p_1
 \end{array}$$

Finalmente, aplicando la regla Rv a $(p_0 \vee \neg p_1)$, obtenemos:

$$\begin{array}{c}
 \neg((p_0 \wedge p_1) \rightarrow \neg(p_0 \vee \neg p_1)) \\
 | \\
 (p_0 \wedge p_1) \\
 | \\
 \neg\neg(p_0 \vee \neg p_1) \\
 | \\
 (p_0 \vee \neg p_1) \\
 | \\
 p_0 \\
 | \\
 p_1 \\
 \swarrow \quad \searrow \\
 p_0 \qquad \neg p_1 \\
 \qquad \qquad \mathbf{x} \\
 \qquad \qquad \mathbf{(cerrada)}
 \end{array}$$

La rama de la derecha está cerrada. La rama de la izquierda está saturada porque todas las fórmulas que figuran arriba de su nodo terminal p_0 fueron utilizadas. El procedimiento se ha completado.

Sea v una valuación tal que $v(p_0) = v(p_1) = 1$. Entonces subiendo por la rama izquierda vemos que v va satisfaciendo las fórmulas que figuran en la misma, en particular el nodo inicial. Luego $\neg\phi$ es satisfacible y por lo tanto ϕ no es una tautología.

Ejemplo 44

Sea $\omega = ((p_0 \rightarrow p_1) \vee (\neg p_2 \rightarrow p_0))$. Sabemos que ω es una tautología si y sólo si $\neg\omega$ es insatisfacible. Como $\neg\omega$ es de la forma de la premisa $R\neg\vee$, vamos a aplicar la regla correspondiente.

A partir de la construcción de un árbol de refutación de la fórmula $\neg\omega$, decidiremos si ω es una tautología o no. Un árbol de refutación para $\neg\omega$ es:

$$\begin{array}{c}
 \neg((p_0 \rightarrow p_1) \vee (\neg p_2 \rightarrow p_0)) \\
 | \\
 \neg(p_0 \rightarrow p_1) \\
 | \\
 \neg(\neg p_2 \rightarrow p_0) \\
 | \\
 p_0 \\
 | \\
 \neg p_1 \\
 | \\
 \neg p_2 \\
 | \\
 \neg p_0 \\
 \mathbf{x} \\
 \text{(cerrada)}
 \end{array}$$

Como la única rama del árbol contiene a la fórmula p_0 y a su negación, no existe valuación que la satisfaga y por lo tanto esta rama es cerrada. Luego $\neg\omega$ es insatisfacible, lo que implica que el árbol tiene todas sus ramas cerradas. Por lo tanto, ω es una tautología.

Hemos visto que los árboles de refutación resultan ser una herramienta muy útil y eficiente cuando debemos decidir sobre la satisfacibilidad de una fórmula del Cálculo Proposicional. Un uso equivalente de estos árboles permite concluir acerca de la validez de un razonamiento.

Ejemplo 45

Sea la siguiente inferencia, dada en el Ejemplo 56 del Capítulo 1. Deseamos analizar mediante el método de los árboles de refutación si dicha inferencia es válida o no.

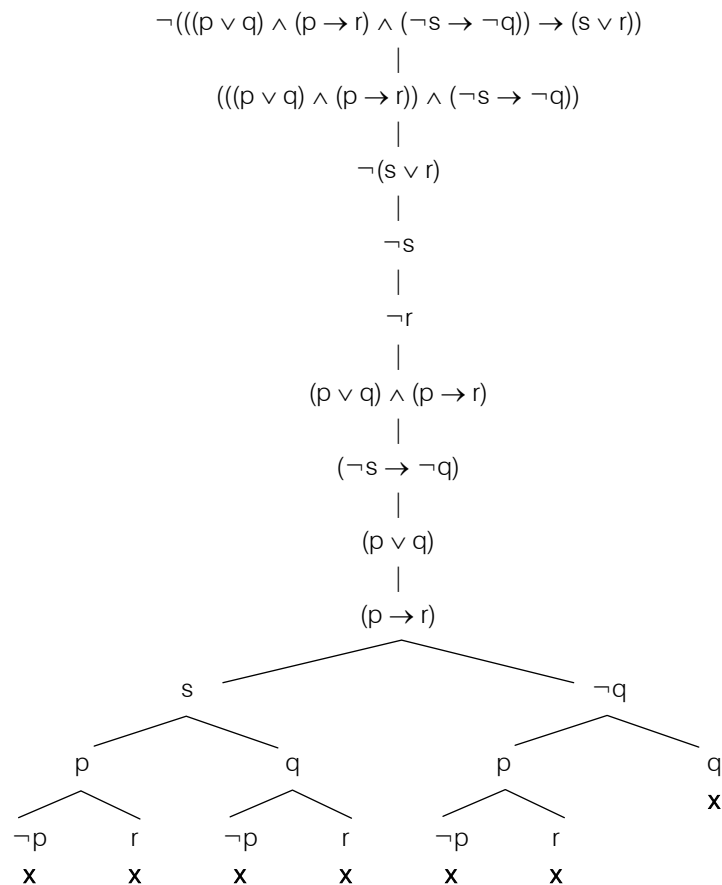
$$\begin{array}{l}
 p_1: (p \vee q) \\
 p_2: (p \rightarrow r) \\
 \underline{p_3: (\neg s \rightarrow \neg q)} \\
 c: (s \vee r)
 \end{array}$$

Este razonamiento es válido si la implicación ϕ dada por:

$$\varphi = (((p \vee q) \wedge (p \rightarrow r) \wedge (\neg s \rightarrow \neg q)) \rightarrow (s \vee r))$$

es tautológica, o equivalentemente si $\neg\phi$ es insatisfacible.

Utilizando las reglas de comportamiento de los conectivos construimos un árbol de refutación completo de $\neg\varphi$:



Todas las ramas del árbol de refutación de $\neg\phi$ son cerradas, con lo cual el árbol es cerrado. Entonces $\neg\phi$ es insatisfacible, o lo que es equivalente ϕ es tautológica o válida. Por lo tanto, podemos concluir que el razonamiento es válido.

Ejemplo 46

Consideremos las siguientes afirmaciones:

θ : La lógica es difícil o a pocos estudiantes les gusta la lógica.

φ : Si las matemáticas son fáciles entonces la lógica no es difícil.

Determinaremos si de θ y φ se deduce lógicamente la siguiente conclusión:

β : Las matemáticas no son fáciles o la lógica es difícil.

En primer lugar debemos determinar las variables proposicionales presentes en las oraciones anteriores. Definimos:

p_1 : La lógica es difícil,

p_2 : A pocos estudiantes les gusta la lógica,

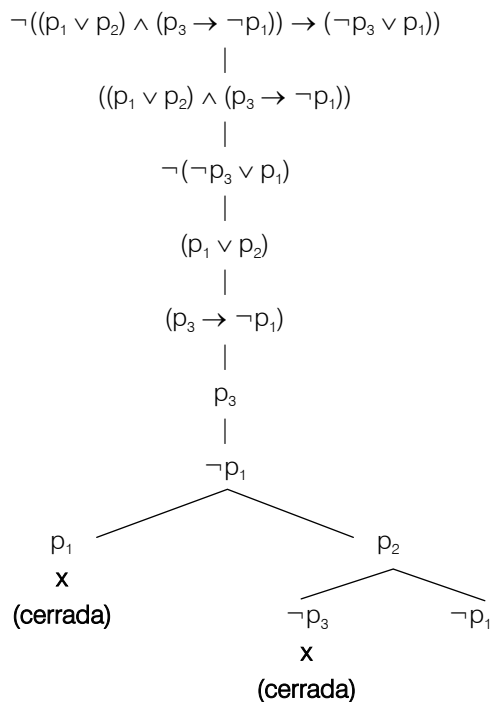
p_3 : Las matemáticas son fáciles.

Entonces $\theta = (p_1 \vee p_2)$, $\varphi = (p_3 \rightarrow \neg p_1)$, $\beta = (\neg p_3 \vee p_1)$.

La fórmula β es consecuencia lógica del conjunto de fórmulas $\{\theta, \varphi\}$, es decir $\{\theta, \varphi\} \models \beta$, si toda valuación que hace verdadera a las fórmulas θ y φ también hace verdadera a β . Por el teorema de la deducción, esto equivale a probar que la implicación $((\theta \wedge \varphi) \rightarrow \beta)$ es tautológica.

Sabemos que $((\theta \wedge \varphi) \rightarrow \beta)$ es una tautología si y sólo si $\neg((\theta \wedge \varphi) \rightarrow \beta)$ es insatisfacible.

Supongamos que $\neg((\theta \wedge \varphi) \rightarrow \beta)$ es satisfacible, entonces debe existir una valuación v que la satisfaga. Veamos si esto es posible mediante la construcción de un árbol de refutación para $\neg((\theta \wedge \varphi) \rightarrow \beta) = \neg((p_1 \vee p_2) \wedge (p_3 \rightarrow \neg p_1)) \rightarrow (\neg p_3 \vee p_1)$.



Como el árbol posee una rama saturada, podemos encontrar una valuación que hace a $\neg((\theta \wedge \varphi) \rightarrow \beta)$ satisfacible. Sea v una valuación tal que $v(p_1) = 0$, $v(p_2) = 1$ y $v(p_3) = 1$, entonces subiendo por la única rama derecha que se encuentra abierta, v satisface las fórmu-

las que figuran en la misma. Luego podemos concluir que $\neg((\theta \wedge \varphi) \rightarrow \beta)$ es satisfacible y por lo tanto $((\theta \wedge \varphi) \rightarrow \beta)$ no es una tautología. Esto significa que β no es consecuencia lógica del conjunto de fórmulas $\{\theta, \varphi\}$.

Actividad 8

Problema N°1: Dadas las siguientes fórmulas o conjuntos de fórmulas de la LPC, decide si son o no satisfacibles utilizando árboles de refutación. Si son satisfacibles, da una valuación que las satisfaga.

- 1.1) $(p \rightarrow (r \rightarrow p)) \rightarrow \neg(q \vee \neg p)$
- 1.2) $(q \rightarrow (q \wedge r)) \vee ((s \wedge \neg s) \vee (p \rightarrow t))$
- 1.3) $\{p \wedge q, \neg p \wedge q\}$
- 1.4) $\{p \wedge q, \neg p \vee q\}$
- 1.5) $\{p \rightarrow q, p \vee q, \neg q\}$
- 1.6) $\{p \rightarrow q, q \rightarrow r, r \rightarrow s, p \rightarrow s\}$
- 1.7) $\{p \rightarrow q, q \rightarrow r, r \rightarrow s, p \wedge \neg s\}$
- 1.8) $\{p \vee q, p \vee (q \wedge r), p \rightarrow \neg r\}$
- 1.9) $\{p \rightarrow q, (p \wedge q) \rightarrow r, q \rightarrow \neg p\}$

Problema N°2: Utilizando árboles de refutación, decide si las siguientes fórmulas son tautologías, contradicciones o contingencias. Para las contingencias halla una valuación que las satisfaga y otra que no, sólo usando la información que dan los árboles.

- 2.1) $\neg(p_1 \vee p_2) \rightarrow ((p_3 \wedge p_1) \vee (p_2 \rightarrow p_3))$
- 2.2) $\neg((p_1 \rightarrow p_3) \rightarrow ((p_2 \rightarrow p_3) \rightarrow ((p_1 \vee p_2) \rightarrow p_3)))$
- 2.3) $((p_1 \rightarrow p_2) \rightarrow p_1) \rightarrow p_1$
- 2.4) $(\neg \neg \neg (p_1 \wedge p_2) \vee p_3) \rightarrow p_4$
- 2.5) $(p_1 \vee \neg p_1) \vee (p_2 \wedge \neg p_2)$

Problema N°3: Usando árboles de refutación, decide si las inferencias dadas son válidas.

- | | |
|----------------------------------|--|
| 3.1) | 3.2) |
| $p_1: \neg p \rightarrow \neg q$ | $p_1: r \wedge t \rightarrow q$ |
| $\underline{p_2: q}$ | $p_2: q \rightarrow s$ |
| $\therefore \neg p$ | $\underline{p_3: \neg s}$ |
| | $\therefore \neg r \vee \neg t$ |
| 3.3) | 3.4) |
| $p_1: p \rightarrow q$ | $p_1: p \rightarrow r$ |
| $\underline{p_2: q}$ | $\underline{p_2: \neg c \rightarrow \neg r}$ |
| $\therefore p$ | $\therefore p \rightarrow c$ |

3.5)

$p_1: p \rightarrow q$

$p_2: r \rightarrow s$

$p_3: p \vee r$

$\therefore q \wedge s$

3.6)

$p_1: p \vee q$

$p_2: p \rightarrow r$

$p_3: q \rightarrow r$

$\therefore r$

Problema N°4: Construye un árbol de refutación completo para cada una de las siguientes fórmulas. Luego, a partir de este árbol construye, cuando sea posible, la forma normal conjuntiva o segunda forma canónica de cada fórmula.

4.1) $(p \rightarrow q) \rightarrow (\neg p \rightarrow \neg q)$

4.2) $((\neg p \rightarrow q) \vee (p \wedge q)) \rightarrow \neg q$

4.3) $((p \wedge (\neg q \vee r)) \rightarrow (\neg r \rightarrow q)) \rightarrow \neg q$

4.4) $(p \rightarrow q) \wedge (\neg p \rightarrow q) \wedge \neg q$

4.5) $((p \rightarrow (q \wedge p)) \vee (\neg q \wedge p))$

Problema N°5: Utilizando árboles de refutación, decide si los siguientes razonamientos son válidos o no.

5.1) Si U es un subespacio de V , entonces U es subconjunto de V , contiene al vector cero y U es cerrado. U es un subconjunto de V y es cerrado, entonces U contiene al vector cero. Así pues, si U es cerrado entonces es un subespacio de V .

5.2) Si Javier ganó la carrera entonces Andrés fue el segundo o Pablo fue el segundo. Si Andrés fue el segundo entonces Javier no ganó la carrera. Si Nicolás fue el segundo entonces Pablo no fue el segundo. Javier ganó la carrera. Luego, Nicolás no fue el segundo.

5.3) Si el color del firmamento no es azul, entonces o está nublado o hay un eclipse. El color es azul o de lo contrario hay un eclipse. Por lo tanto está nublado.

Problema N°6: Determina si los siguientes enunciados son verdaderos o falsos justificando la respuesta.

6.1) Toda rama abierta de un árbol de refutación de una fórmula θ es saturada.

6.2) Si un árbol de refutación completo de la fórmula $(\theta \rightarrow \phi)$ posee todas sus ramas cerradas, entonces $(\theta \wedge \neg \phi)$ es una tautología.

6.3) Si un árbol de refutación completo de una fórmula θ posee todas sus ramas abiertas, entonces θ es una tautología.

6.4) Si un árbol de refutación completo de una fórmula θ posee todas sus ramas abiertas, entonces θ es una contingencia.

6.11 Lectura complementaria

Diseño de conjuntos usando la estructura de árbol Trie

En muchos procesadores de texto actuales existen revisores ortográficos que permiten,

en el momento en que se lo está utilizando, testear que las palabras escritas estén definidas. Para implementar esto se necesita diseñar un conjunto de “palabras” que permita la búsqueda de las palabras válidas en forma muy eficiente. Es en este contexto donde se emplea la estructura de árbol *trie*. Esta estructura, cuyo nombre deriva de la palabra inglesa *retrieval* (recuperación) se utiliza para representar y almacenar conjuntos de cadenas de caracteres definidas sobre un alfabeto Σ .

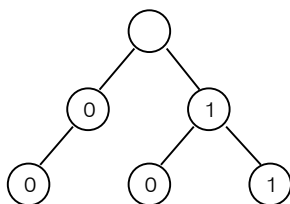
Definición 6.58

Sea Σ un alfabeto de tamaño n . Un trie definido sobre Σ es un árbol n -ario (árbol en el que cada nodo tiene a lo sumo n hijos) donde cada camino desde la raíz a una hoja corresponde a una palabra perteneciente al conjunto representado por ese trie.

En un trie cada nodo excepto la raíz representa un símbolo de un alfabeto dado. Como en un trie la información esta representada por los caminos desde la raíz hasta las hojas, el número de hojas del árbol nos permite determinar la cantidad de elementos que representa ese trie.

Ejemplo 47

Dado el alfabeto $\{0, 1\}$, el siguiente árbol es un trie que representa el conjunto de cadenas $\{00, 10, 11\}$.



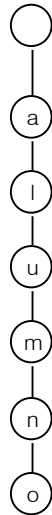
La construcción de un trie se basa en la idea de que todas las palabras almacenadas que tengan en común un prefijo deben tener en común a los nodos que definen el prefijo. Así en el ejemplo 47, podemos observar de la figura que las cadenas 10 y 11 tienen en común el nodo en el nivel 1 que contiene al 1.

Ejemplo 48

Consideremos el siguiente conjunto de cadenas {alumno, aula, camino, casa, clase, claves, crecer}, definidas sobre el alfabeto del idioma castellano $\Sigma = \{a, b, c, \dots, z\}$, donde las mayúsculas y minúsculas se consideran como iguales.

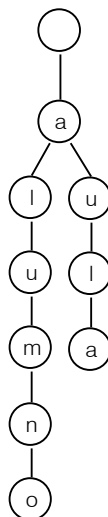
Veamos como construir el trie correspondiente.

Comenzamos con la cadena “alumno”, construimos un árbol en el cual cada letra de la cadena alumno estará representada por un nodo distinto, como lo vemos en la siguiente figura.



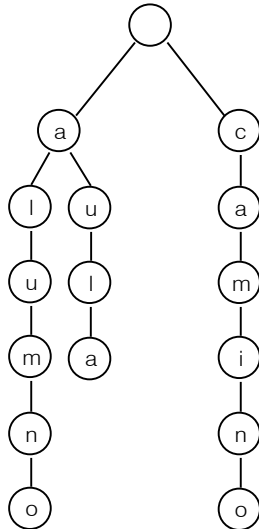
Notemos que en el trie cada nivel a partir del nivel 1 contiene un caracter de la cadena alumno. El primer caracter “a” en el nivel 1, el segundo caracter “l” en el nivel 2, y así sucesivamente.

Ahora debemos añadir al trie la cadena “aula”. Como las cadenas alumno y aula tienen el prefijo 1 en común en el nivel 1, añadimos desde ese nodo una rama para representar los caracteres restantes de la palabra aula. El trie resultante es:

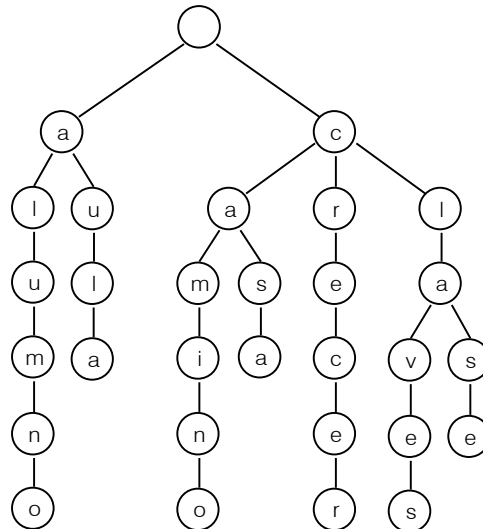


Ahora, corresponde representar la cadena “camino”. Notemos que el primer caracter de esta cadena es c y que no existe en el primer nivel un nodo que contenga dicho caracter, lo

cual significa que no es prefijo de las palabras ya representadas y que por lo tanto debemos añadirlo. Así, representamos dicha cadena como muestra la siguiente figura.



Si añadimos las cadenas restantes: casa, clase, claves, crecer, el trie resultante es:



Ejemplo 49

Veamos como representar los números enteros no negativos menores que ocho en un trie sobre el alfabeto $\{0, 1\}$.

Inicialmente debemos hallar la representación de dichos números en el sistema binario, es decir, como sucesiones de ceros y unos.

Así, por ejemplo:

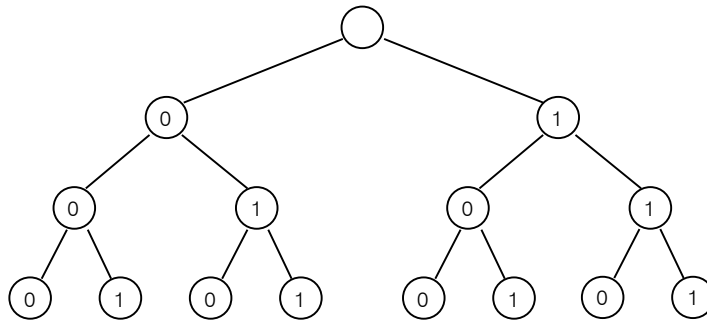
$$0 = 000_2 \text{ porque } 0 = 0 \cdot 2^2 + 0 \cdot 2^1 + 0 \cdot 2^0$$

$$1 = 001_2 \text{ porque } 1 = 0 \cdot 2^2 + 0 \cdot 2^1 + 1 \cdot 2^0$$

.....

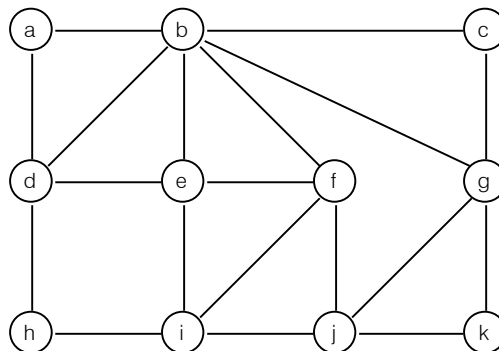
$$7 = 111_2 \text{ porque } 7 = 1 \cdot 2^2 + 1 \cdot 2^1 + 1 \cdot 2^0$$

Luego el conjunto de cadenas que debemos representar con el trie es $\{000, 001, 010, 011, 100, 101, 110, 111\}$



6.12 Problemas complementarios

Problema N°1: Dado el siguiente grafo:



1.1) Halla un ciclo de Euler.

1.2) Si la arista $\{d, e\}$ se elimina, ¿es posible hallar un ciclo de Euler para el grafo parcial resultante?

1.3) Encuentra dos árboles de expansión para este grafo.

Problema N°2: Si $G = (V, A, \phi)$ es un grafo conexo con $|A| = 28$ y $\text{grad}(v) \geq 5$ para todo vértice del grafo. ¿Cuál es el máximo valor para $|V|$?

Problema N°3: Construye el árbol de la siguiente expresión:

$$(+ (- (+ (a, * (b, c)), + (+ (c, d), e)), * (* (e, f), g))).$$

Problema N°4: En un árbol los vértices que no son las hojas tienen respectivamente grados: 2, 1, 3, 3, 2, 3, 4, 4, 4, ¿Cuántas aristas tiene el árbol? ¿Cuántas hojas tiene? ¿Cuántos vértices tiene? Justifica.

Problema N°5: Responde:

5.1) Si G es un grafo de 36 aristas que tiene todos los vértices de grado 4, excepto los 8 vértices pendientes. ¿Cuántos vértices tiene el grafo?

5.2) Sobre 5 vértices, ¿cuántos digrafos (sin arcos estrictamente paralelos) se pueden definir para que sean simétricos? Justifica.

5.3) Determina el número de vértices para los siguientes grafos, justificando la respuesta:

a) G tiene 64 aristas y todos los vértices tienen grado 8.

b) G es un grafo completo más un bucle en cada vértice de 820 aristas.

Problema N°6: Una aerolínea da servicio a cinco ciudades: C_1, C_2, C_3, C_4, C_5 . La siguiente matriz da el costo en pesos del viaje de C_i a C_j :

de	C_1	C_2	C_3	C_4	C_5
C_1		140	100	150	200
C_2	140		200	160	220
C_3	100	200		190	250
C_4	150	160	190		150
C_5	200	220	250	150	

Sobre el conjunto de ciudades se define la relación $R: C_i R C_j$ si y sólo si el viaje entre estas ciudades es menor o igual que \$180.

6.1) Esquematiza la tabla mediante un grafo donde los nodos sean las ciudades y las aristas tengan como etiqueta el costo del traslado entre dos ciudades. Escribe además su matriz de adyacencia.

6.2) Encuentra los pares ordenados que definen la relación R y marca el grafo parcial de la relación.

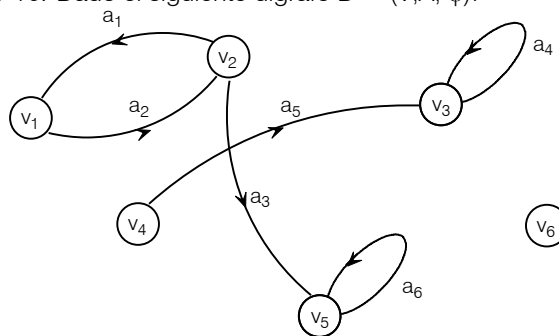
6.3) ¿Qué propiedades tiene este grafo parcial?

Problema N°7: Muestra que la siguiente afirmación no es cierta: "La lista de elementos que se obtiene de recorrer un árbol en preorden es igual a la inversa de la lista que se obtiene al recorrer el árbol en postorden". (La lista inversa de una lista $L = a_1, a_2, a_3, \dots, a_n$ es la lista $L' = a_n, a_{n-1}, a_{n-2}, \dots, a_1$).

Problema N°8: Define un recorrido en orden lexicográfico (usando la definición dada en el Problema N°2 de la Actividad 7) y muestra como se recorre el árbol del Problema N°3 de la Actividad 7.

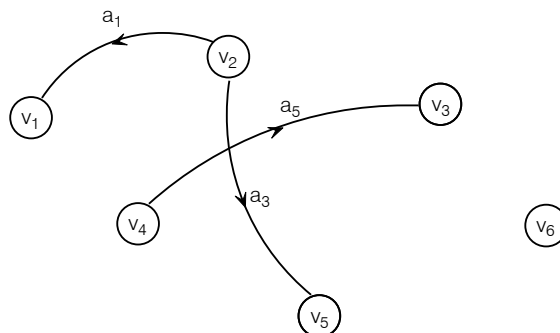
Problema N°9: ¿Cómo tratarías operadores unarios (por ejemplo $+$: positivo, $-$: negativo) para representar expresiones como árboles?

Problema N°10: Dado el siguiente digrafo $D = (V, A, \varphi)$:

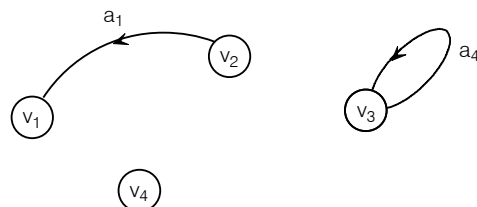


10.1) Clasifica los siguientes digrafos como: digrafo parcial, subdigrafo o subdigrafo parcial de digrafo D , indicando en cada uno de los casos las condiciones que se cumplen.

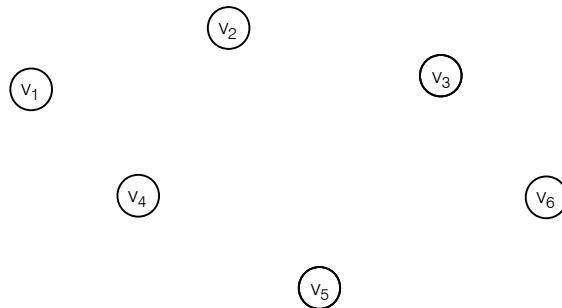
a)



b)



c)



10.2) ¿Cuántos caminos de longitud tres existen de v_1 a v_5 ? Muestra los mismos.

Problema N°11: Lee y Responde:

11.1) ¿Es posible construir un grafo 3-regular con cinco vértices? Justifica.

11.2) ¿Es posible construir un grafo completo de 25 aristas? Justifica.

11.3) Si en un grafo los vértices tienen respectivamente grados: 2, 2, 3, 6, 2, 3, 4, 4, 4, 2; calcula algebraicamente la cantidad de aristas del grafo. Justifica.

Problema N°12: Grafica:

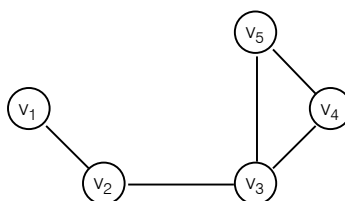
12.1) Un grafo que sea Euleriano y Hamiltoniano.

12.2) Un grafo que sea Euleriano pero no Hamiltoniano.

12.3) Un grafo que sea Hamiltoniano pero no Euleriano.

12.4) Un grafo Conexo acíclico.

Problema N°13: Dado el siguiente grafo $G = (V, A, \phi)$, dibuja todos los árboles de expansión para dicho grafo.



Problema N°14: Muestra por inducción que el número de nodos de grado dos de cualquier árbol binario es uno menos que el número de niveles.

Problema N°15: Lee y responde:

15.1) ¿Qué altura debería tener un trie si se pretende representar el número 278, sobre el alfabeto $\{0, 1\}$?

15.2) ¿Cuántos elementos se pueden representar sobre el alfabeto $\{0, 1\}$ con un trie de altura n ?

15.3) ¿Cuántas cadenas de longitud 3 se pueden representar con un trie sobre el alfabeto $\{a, b, c\}$?

6.13 Ejercicios de opción múltiple

Ejercicio 1: Sea T un árbol con raíz v_0 . Señala la proposición que es verdadera:

- a) T puede tener circuitos.
- b) Todo vértice de T tiene grado negativo igual a 1.
- c) Todo vértice tiene grado positivo mayor o igual que 1.
- d) Ninguna de las anteriores.

Ejercicio 2: Sea $G = (V, A, \varphi)$ un grafo conexo. Señala la proposición que es verdadera:

- a) $o(A) > o(V) - 1$.
- b) Existe siempre una arista entre dos vértices distintos.
- c) $o(A) = o(V)$.
- d) Ninguna de las anteriores.

Ejercicio 3: Sea $G = (V, A, \varphi)$ un grafo sin lazos. Si $g(v_i) \geq (n - 1)/2$ para todo v_i de V , entonces podemos afirmar que:

- a) G es conexo con la mínima cantidad de aristas posibles.
- b) G tiene un camino de Hamilton.
- c) G es regular.
- d) Ninguna de las anteriores.

Ejercicio 4: Sea K_n un grafo completo de n vértices y a aristas, luego podemos afirmar que:

- a) $a = n^2 - n$.
- b) $g(v_i) = n$ para todo v_i de V .
- c) K_n es euleriano si n es un número impar.
- d) Ninguna de las anteriores.

Ejercicio 5: Sea T un árbol con raíz v_0 . Señala cuál de las siguientes afirmaciones es verdadera:

- a) T es árbol binario si todos los vértices internos tienen grado positivo igual a 2.
- b) Solo los vértices pendientes tienen grado negativo igual a 1.
- c) Existe un vértice de grado negativo igual a cero.
- d) Ninguna de las anteriores.

Ejercicio 6: Sea $G = (V, A, \varphi)$ un grafo conexo, este admite un ciclo euleriano si y solo si:

- a) Existe al menos un vértice de G con grado par.
- b) Todos los vértices de G tienen grado par.
- c) $o(A) = o(V) - 1$.
- d) Ninguna de las anteriores.

Ejercicio 7: Sea G un grafo cuya matriz de adyacencia está dada por:

$$M(G) = \begin{bmatrix} 0 & 1 & 1 & 0 & 1 \\ 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 0 & 1 & 1 \\ 0 & 0 & 1 & 0 & 1 \\ 1 & 0 & 1 & 1 & 0 \end{bmatrix}$$

Señala cuál de las siguientes afirmaciones es verdadera:

- a) G es euleriano.
- b) G es regular.
- c) G tiene un nodo de grado 4.
- d) Ninguna de las anteriores.

Ejercicio 8: Si $(1 (2 (3, \lambda), 4 (5 (\lambda, 7), \lambda)))$ es la notación prefija usando paréntesis anidados de un árbol, entonces su recorrido en postorden es:

- a) 7 5 4 3 2 1
- b) 3 2 7 5 4 1
- c) 3 2 1 5 7 4
- d) Ninguna de las anteriores.

Ejercicio 9: En un árbol binario de altura 20 hay:

- a) 220 hojas.
- b) 20 nodos internos.
- c) Por lo menos 21 nodos.
- d) Ninguna de las anteriores.

Ejercicio 10: La expresión $f a b + * c d * e +$

- a) corresponde a la notación postfija de un árbol con raíz $*$.
- b) corresponde a la notación postfija de un árbol con raíz $+$.
- c) no corresponde a un árbol.
- d) corresponde a la notación postfija de un árbol de altura 3.

Ejercicio 11: Una valuación que satisface a la fórmula $\phi = \neg ((p \rightarrow q) \vee (\neg r \rightarrow p))$ es:

- a) $v(p) = 0, v(q) = 0, v(r) = 0$.
- b) $v(p) = 0, v(q) = 1, v(r) = 0$.
- c) $v(p) = 1, v(q) = 1, v(r) = 0$.

d) Ninguna de las anteriores.

Ejercicio 12: Si todo árbol de refutación completo de la fórmula $((\theta \rightarrow \varphi) \wedge \beta)$ posee todas sus ramas cerradas, entonces:

- a) $((\theta \rightarrow \varphi) \wedge \beta)$ es una tautología.
- b) $((\theta \rightarrow \varphi) \rightarrow \neg \beta)$ es una tautología.
- b) $(\theta \rightarrow \varphi)$ es una contradicción.
- d) Ninguna de las anteriores.